

A
Major Project Report
On
CampusConnect: - College ERP Portal
For the Partial Fulfillment of the Award of
Degree Of
Masters of Computer Applications (MCA)



UNDER THE GUIDANCE OF:

Mr. Amit Jha
Assistant Professor

SUBMITTED BY:

Jai soni
23MCAN0022

School of Computer Applications
JECRC UNIVERSITY, JAIPUR
2024-2025

Candidate's Declaration

I hereby declare that the project work, which is being presented in the Project Report, entitled '***CampusConnect: - College ERP Portal***' in partial fulfillment for the award of Degree of "**Master of Computer Application**" in School of Computer Application, **JECRC University** is a record of my own investigations carried under the guidance of **Mr. Amit Jha**. I have not submitted the matter presented in this Project Report anywhere for the award of any other Degree.

Name of Candidates: Jai Soni

Signature:

Registration No: 23MCAN0022

Name(s) of Supervisor(s)/Guide:

Mr. Amit Jha

Countersigned By: HOD- IT & CA

JECRC University
School of Computer Applications
Major Project Completion Certificate

This is to certify that **Jai soni** Registration Number **23MCAN0022** has successfully completed the Major Project as a part of the requirements for the Masters of Computer Applications (MCA) Program at JECRC University, Jaipur, India.

Project Title: CampusConnect: - College ERP Portal

Technology Used: HTML, CSS, JavaScript, Bootstrap, Python(Django), SQLite

Project Supervisor: Mr. Amit Jha

This project demonstrates student's ability to apply theoretical and practical knowledge gained during his/her academic studies to practical real-world scenarios. His / Her dedication, hard work, and proficiency in the subject matter is commendable.

A handwritten signature in black ink, appearing to read 'Shekhar', is displayed on a light gray rectangular background.

Mr. Shekhar Chander
HOD

Project Supervisor

ACKNOWLEDGEMENT

With Candor and pleasure, I take the opportunity to express my sincere thanks and obligation to my esteemed guide. It is because of his able and mature guidance and co-operation without which it would not have been possible for me to complete my project.

It is my pleasant duty to thank all the staff members of the computer center who never hesitated to help me from time during the project.

Finally, I gratefully acknowledge the support, encouragement, and patience of my family and as always, nothing in my life would be possible without God.

Thank You!

TABLE OF CONTENT

1. Problem Definition	
1.1 Introduction	1
1.2 Need and Purpose	1-2
2. Objective	2-3
3. System Study	
3.1 Feasibility Study	3
3.1.1 Economic Feasibility	3
3.1.2 Operational Feasibility	3-4
3.1.3 Technical Feasibility	4
3.1.4 Motivational Feasibility	4
3.1.5 Schedule Feasibility	4-5
3.2 Cost and Benefit Analysis	5
3.3 Fact Finding Techniques	5
3.4 System Overview	5-7
4. System Analysis	
4.1 College ERP Portal	7-8
4.2 Function Details	8-10
4.3 Testing	10-11
4.4 Functional and Non-Functional Requirements	11-12
5. System Development Cycle	
5.1 Context Level DFD	13-14
5.2 User Interface (Admin)	14
5.3 User Interface (Staff)	15
5.4 User Interface (Student)	15
6. Data Dictionary	
6.1 System Design	16-17
6.2 Scheduling	17
6.3 Estimates of Effort	18
6.4 Modules of CampusConnect	18-19
7. Source Code	20-77
8. Limitations and Future Enhancements	77-79
9. Conclusion	79-80
10. Bibliography	80-81

1. Problem Definition

1.1 Introduction

Educational institutions, regardless of their size, face significant challenges when it comes to managing and maintaining records of students and staff. Traditionally, many of these processes are carried out manually, involving paper-based forms or disconnected software solutions. These outdated methods can lead to inefficiencies, such as difficulty in tracking student progress, errors in staff scheduling, and slow response times for essential administrative functions. Furthermore, these manual systems increase the risk of data loss, making it harder to retrieve or update vital information.

This project seeks to address these challenges by developing a comprehensive College ERP Portal, which will provide a centralized and automated solution for academic and administrative tasks. The College ERP Portal will not only streamline processes but also create a more efficient, accurate, and accessible environment for students, staff, and administrators alike. By integrating all crucial tasks into a single digital platform, this system will significantly reduce the manual workload while ensuring seamless communication and data management across the institution.

1.2 Need and Purpose

Need of the Project:

- **Inefficiency of Manual Systems:** Educational institutions face challenges with manual systems, leading to errors, delays, and difficulty in retrieving/updating information.
- **Fragmented Information:** Data is often spread across multiple disconnected systems, making it hard to track and manage student and staff records effectively.
- **Time-Consuming Processes:** Manual handling of administrative tasks such as attendance and leave management wastes time and resources.
- **Data Security and Access Issues:** Manual systems are prone to data loss and unauthorized access, increasing risks to sensitive student and staff information

- **Communication Barriers:** Lack of a unified platform leads to poor communication and delays in important updates between administrators, staff, and students.

Purpose of the Project

The purpose of the College ERP Portal is to streamline and automate academic and administrative processes within educational institutions. By digitizing the management of student and staff data, the portal aims to eliminate the inefficiencies and errors associated with traditional manual systems. The system is designed to centralize critical information, making it easily accessible to authorized users and ensuring that all tasks, such as attendance tracking, leave approvals, and result processing, are handled efficiently. Through this automation, the portal reduces the burden on administrative staff, allowing them to focus on more strategic activities.

Additionally, the portal seeks to improve communication within the institution. It offers a unified platform for notifications, updates, and alerts, ensuring that students, staff, and administrators are always informed about important events, deadlines, and changes. The College ERP Portal also ensures data security by providing role-based access control, protecting sensitive information from unauthorized access. Ultimately, the portal's goal is to enhance operational efficiency, improve decision-making, and foster a more organized, transparent, and responsive academic environment.

2. Objective

The primary objective of this project is to develop a **College ERP Portal** that streamlines the management of academic and administrative tasks within educational institutions. Traditional methods of managing student and staff records, attendance, results, and other processes often involve manual effort, paper-based records, or disparate systems that are inefficient and prone to errors. By providing a centralized digital solution, this project aims to address these inefficiencies and enable institutions to operate more effectively.

The portal will offer a user-friendly interface for three main user groups—Admins, Staff, and Students—ensuring that each group has access to

functionalities tailored to their roles. Admins will be able to manage core activities such as adding or updating staff, students, courses, and sessions. Staff members can handle responsibilities like taking attendance, processing results, and applying for leave. Students, on the other hand, can view attendance, submit feedback, apply for leave, and access library resources. This role-based access ensures secure data handling while improving the overall user experience.

Additionally, the project aims to enhance communication between stakeholders by integrating features like notifications and real-time chat. The automation of tasks such as attendance tracking, leave management, and result generation will minimize manual errors, save time, and improve accuracy. Ultimately, the College ERP Portal aspires to not only simplify daily operations but also support informed decision-making through comprehensive data management and reporting tools. By digitizing institutional processes, the project aims to foster a culture of efficiency, transparency, and continuous improvement in educational institutions.

3. System Study

3.1 Feasibility Study

3.1.1 Economic Feasibility

- The project requires minimal investment in terms of development tools since most are open-source (e.g., Django, Python).
- Long-term cost savings by reducing paper-based processes and manual errors.

3.1.2 Operational Feasibility

- Easy-to-use interfaces for all user roles (Admin, Staff, Students).
- Comprehensive features cater to the day-to-day needs of colleges, ensuring

widespread adoption.

3.1.3 Technical Feasibility

- Built using robust and scalable technologies like Django, HTML, CSS, and JavaScript.
- Secure data handling with role-based access control.
- Easily deployable on existing college IT infrastructure

The chosen technologies are reliable, well-documented, and widely adopted, ensuring technical feasibility.

3.1.4 Motivational Feasibility

- Encourages digitization and automation, reducing staff workload.
- Enhances student and staff satisfaction by improving access to academic and administrative information.

3.1.5 Schedule Feasibility

Schedule feasibility assesses whether the project can be completed within the available time frame. A detailed development timeline ensures all phases, from requirement gathering to deployment, are completed on time.

Development Timeline:

1. **Weeks 1-2:** Requirement gathering and system design.
2. **Weeks 3-6:** Development of core functionalities (frontend and backend).
3. **Weeks 7-8:** Integration and system testing.
4. **Week 9:** Final deployment and review.

The *CampusConnect* project is designed to meet its deadlines for the initial release, with core features like user authentication, update profile, view attendance, apply for leave

The project's schedule feasibility is high, as the team can effectively allocate time

for each development phase, ensuring a smooth launch and consistent updates to improve the application over time.

3.2 Cost and Benefit Analysis

Cost and benefit analysis highlights the financial viability of *CampusConnect* by comparing development costs to the anticipated benefits.

Costs:

- Developer time and effort.
- Minimal expenses for testing and tools.

Benefits:

- Long-term savings through open-source technologies.
- Increased user engagement with a feature-rich application.
- Scalability to accommodate future growth.

3.3. Fact-Finding Techniques

For the *CampusConnect* project, the fact-finding process was carried out through a variety of methods to understand user requirements, gather market insights, and refine the application's features and functionalities.

- **Questionnaires:** Distributed among users to identify pain points and preferred features.
- **Observation:** Analyzed existing systems to pinpoint inefficiencies.
- **Document Analysis:** Reviewed institutional records to identify data management needs.

3.4. System Overview

CampusConnect is a next-generation chat application designed to offer secure, real-time communication with a wide range of features tailored to modern user

needs. The application focuses on providing a seamless user experience through a robust technology stack and intuitive interface.

Key Features of CampusConnect

- **Admin Panel:** Manages staff, students, courses, subjects, sessions, and sends notifications.
- **Staff Panel:** Handles attendance, result processing, leave requests, and feedback submission.
- **Student Panel:** Views attendance, applies for leave, accesses library books, and submits feedback.

Technology Used:

1. Frontend Technologies:

- **HTML:** To create the structure of web pages.
- **CSS:** For styling the user interface and enhancing the design.
- **JavaScript:** To implement dynamic functionalities and interactivity.
- **Bootstrap:** For responsive design and pre-styled components.

2. Backend Technologies:

- **Python (Django Framework):**
 - Django is used for server-side development, routing, and handling logic.
 - Built-in ORM (Object-Relational Mapping) simplifies database operations.
- **Django Templates:** For rendering dynamic content on web pages.

3. Database:

- **SQLite:** Lightweight and efficient database used for data storage and retrieval.

System Benefits

1. Accessibility:

- is designed for cross-platform compatibility, ensuring usability on desktop, tablet, and mobile devices.

2. Security:

- End-to-end encryption and secure authentication mechanisms protect user data and privacy.

3. Scalability:

- The architecture supports growing user bases and increasing message volumes, ensuring consistent performance.

CampusConnect combines cutting-edge technology with user-focused design. By providing essential features, maintaining security, and planning for future enhancements, *CampusConnect* positions itself as a leading communication platform for the digital age.

4. System Analysis

CampusConnect: College ERP Portal is designed to streamline the management of student and staff information for educational institutions. The portal ensures that accurate, up-to-date records are maintained, facilitating the generation of reports, tracking attendance, and managing academic details. With a user-friendly and secure interface, it provides a comprehensive solution for managing all aspects of student and faculty life within the college.

4.1 College ERP Portal Overview

CampusConnect: College ERP Portal is designed to simplify and automate the management of academic and administrative tasks for educational institutions. The system provides a centralized platform for managing student and staff information, attendance, grades, notifications, and more. By digitizing these tasks, the portal helps reduce manual workload, improves data accuracy, and fosters better

communication among all stakeholders—administrators, staff, and students.

Core Elements of the CampusConnect System:

1. Centralized Data Management:

- All student and staff data is stored in a single, centralized database, ensuring easy access and management for authorized users.
- Role-based access controls ensure data security, allowing different levels of permission for administrators, staff, and students.

2. Attendance and Leave Management:

- The portal automates attendance recording, reducing the manual effort of tracking student presence.
- Students can apply for leave, and staff can approve or reject applications, all tracked through the system.

3. Grade and Result Management:

- Academic results are recorded and updated in real-time, making it easy to track student progress.
- Faculties can upload grades for assignments, quizzes, and exams.

4. Notifications and Alerts:

- The system sends automatic notifications for deadlines, attendance updates, result announcements, and more, ensuring everyone stays informed.

5. Reports and Analytics:

- The portal generates dynamic reports for students, staff, and overall institutional performance, enabling administrators to make data-driven decisions.

6. Cross-Platform Compatibility:

- The portal is designed to be responsive, ensuring access from desktops, smartphones, and tablets for both students and staff.

4.2 Function Details

The *CampusConnect: College ERP Portal* provides a robust set of functionalities

designed to enhance the efficiency of educational institutions. Below is an in-depth description of the key functionalities:

1. **User Authentication:**

- **Registration:** New users can create accounts by providing details such as email, phone number, and password. Passwords are securely stored using encryption.
- **Login/Logout:** Secure login ensures that only authorized users can access the portal. Persistent sessions improve convenience.
- **Multi-Factor Authentication (MFA):** An extra layer of security is added through OTP or email-based verification.

2. **Attendance Management:**

- **Automated Attendance:** Staff can mark attendance for students in real-time during lectures.
- **Leave Application:** Students can request leave, and faculty members can approve or deny it based on predefined policies.

3. **Result Management:**

- **Result Entry:** Faculty members can enter grades for various assessments such as exams and assignments.
- **Student Access:** Students can access their grades in real-time, improving transparency.

4. **Notifications System:**

- **Real-Time Notifications:** Administrators and staff can send updates regarding announcements, deadlines, and important events.
- **Alerts:** Students and staff receive alerts for tasks such as upcoming exams, holidays, or changes in schedules.

5. **Report Generation:**

- **Custom Reports:** The system can generate reports on various aspects like student attendance, academic performance, and faculty assignments.

- **Analytics:** Data analytics tools provide insights into student performance and institutional operations.

6. Cross-Platform Support:

- **Device Accessibility:** The portal supports web access via desktops, tablets, and smartphones, ensuring connectivity at all times.

4.3 Testing

Testing ensures that the *CampusConnect* ERP Portal functions as expected under real-world conditions. The system undergoes comprehensive testing at various stages of development to ensure reliability, security, and performance.

Types of Testing:

1. Unit Testing:

- Focuses on testing individual components such as the login system, attendance marking, or result submission forms.
- Ensures that each module functions correctly in isolation.

2. Integration Testing:

- Verifies the smooth interaction between different modules such as attendance management, leave requests, and student performance records.
- Ensures that data flows correctly between the front-end and back-end systems.

3. System Testing:

- Involves end-to-end testing of the entire system, validating that all modules work together seamlessly.
- Includes testing scenarios such as registration, attendance marking, result uploads, and notifications.

4. Performance Testing:

- Evaluates how the portal performs under high traffic, such as during exam result releases or holiday schedules.
- Stress testing ensures that the system can handle a large number of

concurrent users.

5. Security Testing:

- Focuses on identifying vulnerabilities in data access, user authentication, and data storage.
- Ensures that sensitive data is protected with encryption and secure access controls.

6. User Acceptance Testing (UAT):

- End-users (administrators, faculty, and students) test the system to ensure it meets real-world expectations.
- Collects feedback for further improvements and user satisfaction.

4.4 Functional and Non-Functional Requirements

Functional Requirements:

1. User Registration and Login:

- Secure login with password encryption and multi-factor authentication.

2. Attendance Management:

- Automatic attendance tracking and leave request/approval system.

3. Result Management:

- Ability to enter, update, and access academic results in real-time.

4. Notifications:

- Real-time alerts for deadlines, events, and important announcements.

5. Report Generation:

- Customizable reports for students, staff, and administrative purposes.

6. Cross-Platform Access:

- Support for desktops, tablets, and smartphones.

Non-Functional Requirements:

1. Performance:

- The system must handle at least 5,000 concurrent users without noticeable delays.

- Reports and data updates should be delivered within 2 seconds under normal load.

2. Scalability:

- The system should scale seamlessly as the number of students and staff grows.

3. Security:

- Implement role-based access control to restrict unauthorized access to sensitive data.
- Ensure data encryption and secure transmission of information.

4. Reliability:

- 99.9% uptime, with robust error handling and automatic backups.

5. Usability:

- The user interface should be intuitive and easy to navigate for all user roles.

6. Maintainability:

- The system should be designed with modularity for ease of updates, bug fixes, and feature expansions.

The system analysis of *CampusConnect: College ERP Portal* highlights its well-organized structure, core functionalities, and extensive testing strategies. By focusing on real-time communication, secure data handling, and user-friendly features like attendance and result management, the portal ensures an efficient and effective environment for students, staff, and administrators alike. Rigorous testing guarantees that the system can handle various real-world conditions, while the clearly defined requirements ensure scalability, security, and overall system reliability.

5. System Development Cycle

The System Development Cycle for the CampusConnect: College ERP Portal involves multiple phases, including planning, designing, developing, testing, deploying, and maintaining the system. Throughout these phases, Data Flow

Diagrams (DFDs) are utilized to illustrate the flow of information within the system, ensuring clarity and consistency in development. The following sections provide the context-level DFD and detailed DFDs for specific processes, such as user authentication and attendance management.

5.1 Context-Level Data Flow Diagram (DFD)

The Context-Level DFD offers a high-level overview of the CampusConnect: College ERP Portal. It represents the system as a single process interacting with external entities such as students, staff, and administrators. The diagram demonstrates the flow of data between these entities and the portal.

DFD for User Authentication Process

Entities Involved:

1. User: The user initiates authentication by providing credentials, such as a username and password, through the portal interface.
2. Server: Validates the credentials and facilitates secure access to the system.
3. Database: Stores encrypted user data and session tokens.

Flow of Process:

- Input: The user enters login credentials (username and password).
- Processing:
 - The server encrypts the credentials and verifies them against the data stored in the User Database.
 - If valid, a session token is generated and returned to the user for secure access.
 - If invalid, an error message is displayed, prompting the user to re-enter the correct details.
- Output:
 - A successful login grants the user access to role-specific features within the portal.
 - Failed attempts are logged for security monitoring.

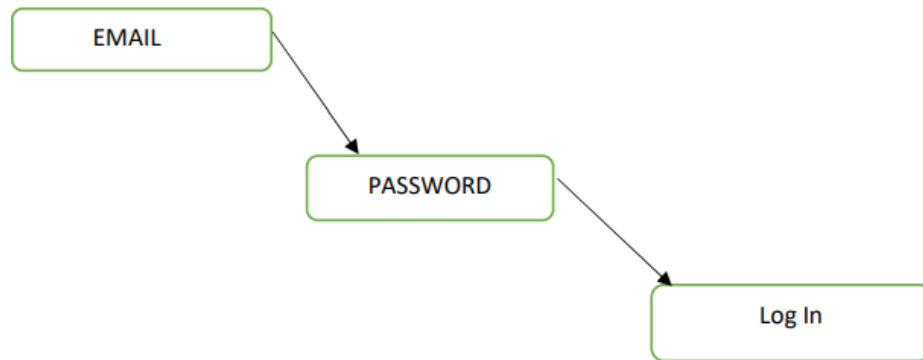


Fig 5.1

In this DFD, the user enters their login details, which are validated and securely stored. Upon successful validation, the user gains access to the app's features.

5.2 User Interface (Admin)

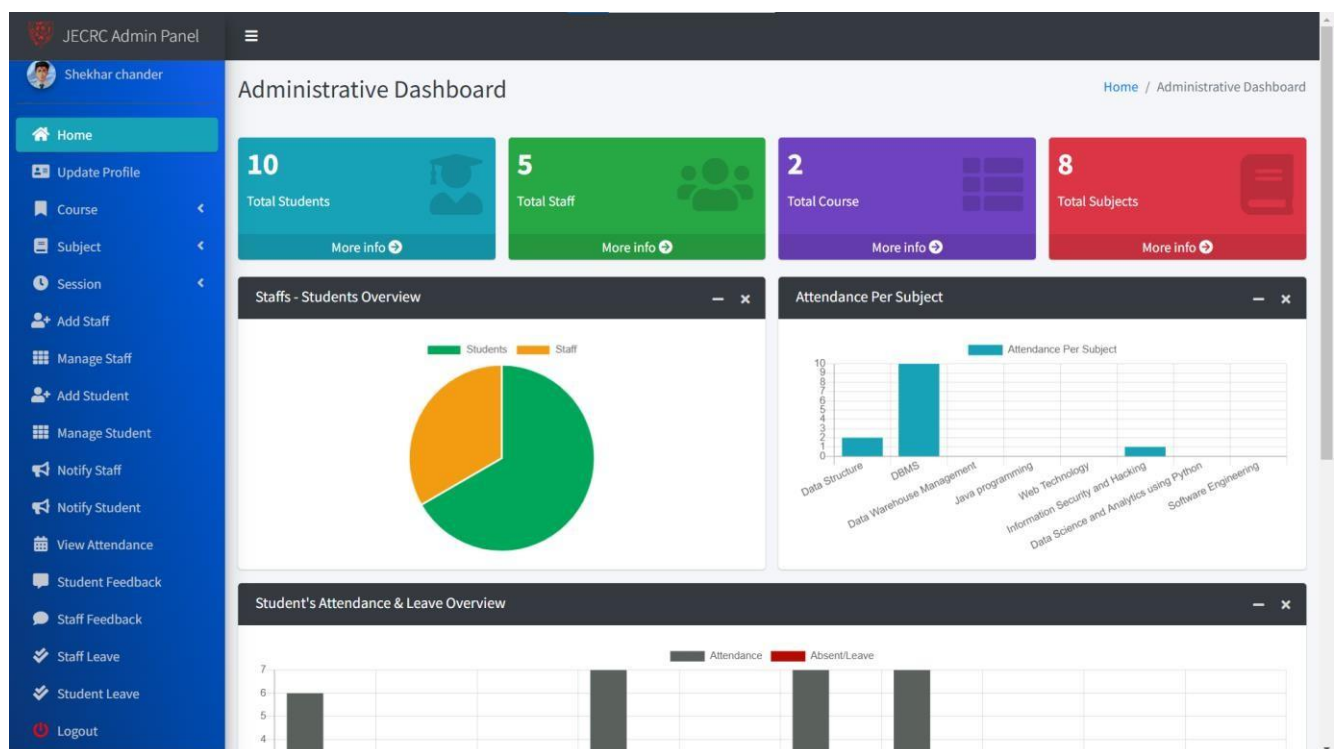


Fig 5.2

5.3 User Interface (Staff)

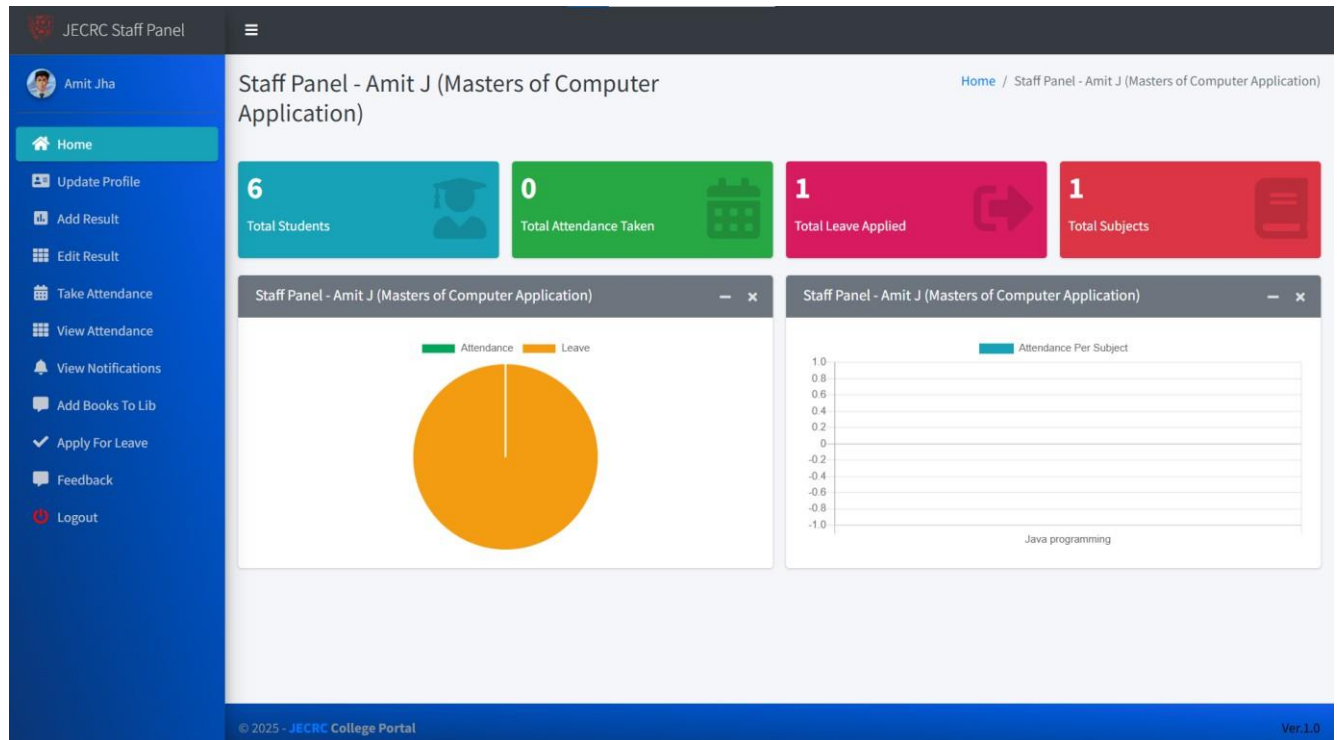


Fig 5.3

5.4 User Interface (Student)

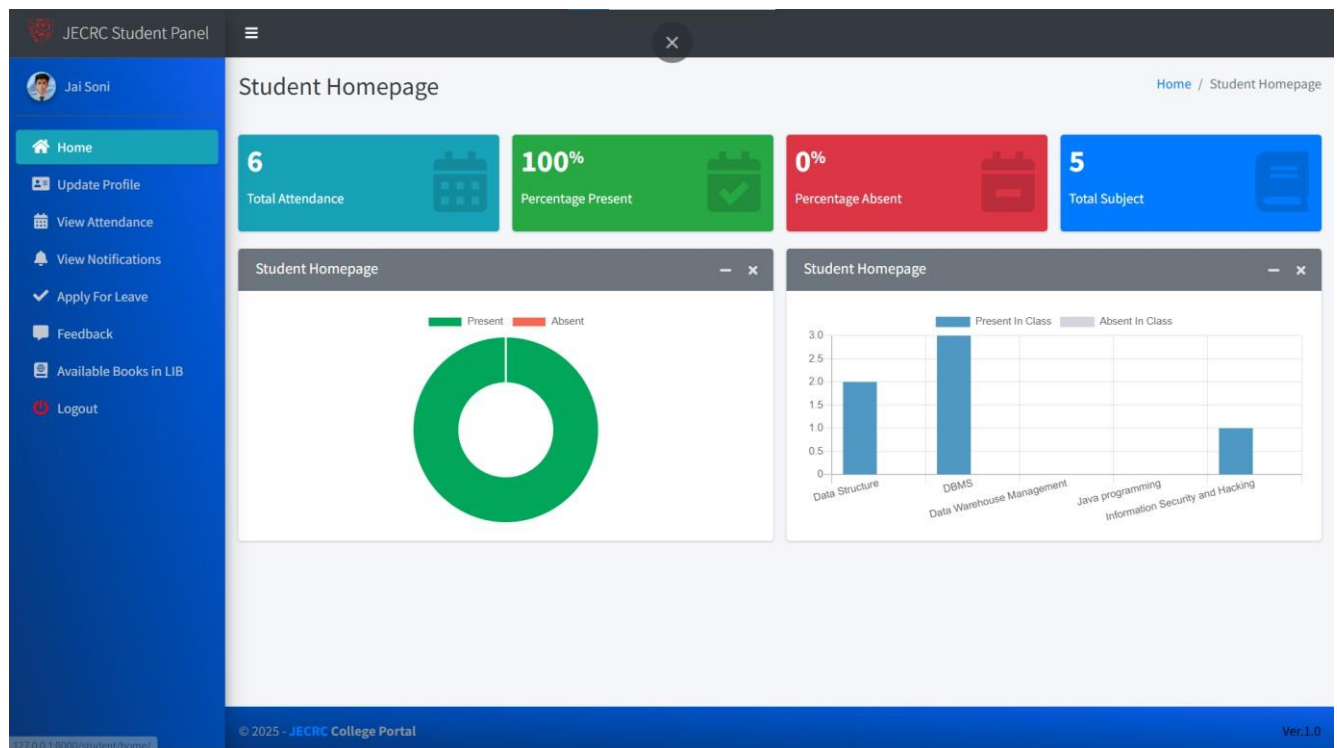


Fig 5.

6. Data Dictionary

The Data Dictionary for *CampusConnect* offers a comprehensive overview of the system's design, scheduling, effort estimation, and modular components. This structured description ensures clarity and serves as a reference for understanding the system's architecture and development.

6.1 System Design

The system design focuses on building a scalable, secure, and efficient architecture for *CampusConnect*.

1. SystemArchitecture:

CampusConnect utilizes a client-server architecture, consisting of:

- **Client-Side (Frontend):**
 - Built using **HTML, CSS, JavaScript, and Bootstrap** to deliver an intuitive and responsive user interface.
 - Communicates with the backend through RESTful APIs for data retrieval, authentication, and form submissions.
- **Server-Side (Backend):**
 - Developed using **Django**, handling user authentication, session management, attendance updates, notifications, and data queries.
 - Implements business logic to process user requests and ensures secure data handling.
- **Database:**
 - **PostgreSQL** is used for reliable and efficient data storage, managing information such as student details, attendance records, course schedules, and notifications.

2. Modules:

- **Authentication Module:** Manages user login, registration, and role-based access.
- **Attendance Module:** Tracks and updates attendance records for students.
- **Course Management Module:** Handles student enrollment, curriculum tracking, and faculty assignments.
- **Notification Module:** Sends announcements and updates to students and staff.

6.2 Scheduling

The development of *CampusConnect* follows a structured timeline:

1. Requirement Gathering and Analysis (2 weeks):

- Collect institutional requirements and define the project scope.

2. System Design (3 weeks):

- Create database schemas, user interface mockups, and workflows.

3. Development and Implementation (6 weeks):

- Build the frontend using **HTML/CSS/Bootstrap** and backend with **Django**.
- Integrate APIs for attendance, notifications, and course management.

4. Testing and Debugging (3 weeks):

- Conduct unit, integration, and user acceptance testing to ensure functionality.

5. Deployment and Maintenance (2 weeks):

- Deploy the application and provide training for end-users.
- Monitor performance and address any issues.

Total Project Duration: Approximately 16 weeks.

6.3 Estimates of Effort

The effort estimation for *CampusConnect* is based on task complexity and resource requirements:

- **Requirement Analysis:** ~12 hours for understanding and documenting institutional needs.
- **System Design:** ~30 hours for architecture planning and database schema creation.
- **Frontend Development:** ~40 hours for building responsive interfaces.
- **Backend Development:** ~70 hours for implementing Django logic and integrating APIs.
- **Database Setup:** ~25 hours for designing and configuring PostgreSQL.
- **Testing and Debugging:** ~35 hours for identifying and resolving issues.
- **Deployment and Review:** ~20 hours for deploying and monitoring the system.

Total Estimated Effort: ~232 hours.

6.4 Modules

The *CampusConnect* system is divided into key functional modules:

1. Authentication Module:

- **Function:** Manages secure login, registration, and role-based access control.
- **Key Features:**
 - Encrypted password storage.
 - Multi-factor authentication (optional).

2. Attendance Module:

- **Function:** Tracks and updates student attendance.
- **Key Features:**
 - Daily and bulk attendance updates.
 - Attendance summaries for faculty and students.

3. Course Management Module:

- **Function:** Handles course allocation and student enrollment.
- **Key Features:**
 - Faculty assignment to courses.
 - Real-time updates to course materials.

4. Notification Module:

- **Function:** Sends updates and announcements.
- **Key Features:**
 - Role-specific notifications.
 - Delivery tracking for announcements.

5. Student and Staff Dashboard Module:

- **Function:** Provides personalized access to relevant resources.
- **Key Features:**
 - To-do lists, academic progress tracking, and access to study materials.

6. Reports and Analytics Module:

- **Function:** Generates detailed reports for administration.
- **Key Features:**
 - Attendance, performance, and course progress analytics.

The data dictionary outlines the detailed structure and planning of *CampusConnect*, covering system design, phased scheduling, estimated efforts, and modular development. This organized approach ensures that the project remains manageable, scalable, and ready for future enhancements.

7. Source Code

*****Login.html*****

```
{% load static %}
<!DOCTYPE html>
<html>

<head>
  <meta charset="utf-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <title>JECRC University Portal</title>
  <!-- Tell the browser to be responsive to screen width -->
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <!-- favicon -->
  <link rel="shortcut icon" href="{% static '/image/favicon.ico' %}">
  <!-- Font Awesome -->
  <link rel="stylesheet" href="{% static 'plugins/fontawesome-free/css/all.min.css' %}">
  <!-- Ionicons -->
  <link rel="stylesheet" href="{% static
'https://code.ionicframework.com/ionicons/2.0.1/css/ionicons.min.css' %}">
  <!-- icheck bootstrap -->
  <link rel="stylesheet" href="{% static 'plugins/icheck-bootstrap/icheck-bootstrap.min.css' %}">
  <!-- Theme style -->
  <link rel="stylesheet" href="{% static 'dist/css/adminlte.min.css' %}">
  <!-- Google Font: Source Sans Pro -->
  <link href="https://fonts.googleapis.com/css?family=Source+Sans+Pro:300,400,400i,700"
rel="stylesheet">
  <!-- Google Recaptcha -->
  <script src="https://www.google.com/recaptcha/api.js" async defer></script>
</head>

<body class="hold-transition login-page" style="background-image: url('{% static 'dist/img/ju-bg.png'
%}'); background-size: cover; background-position: center;">
  <div class="login-box">
    
    <div class="login-logo">
      <a href="" style="color: rgb(0, 0, 0);"><b>JECRC</b><b> University</b></a>

    </div>
    <!-- /.login-logo -->
    <div class="card">
      <div class="card-body login-card-body">
        <!-- <p class="login-box-msg">Sign in to start your session</p> -->
        {% if messages %}
```

```

<div class="col-12">
  {% for message in messages %}
  {% if message.tags == 'error' %}
  <div class="alert alert-danger text-center ">
    {{message }}
  </div>
  {% endif %}
  {% endfor %}
</div>
{% endif %}
<form action="doLogin/" method="post">
  {% csrf_token %}
  <div class="input-group mb-3">
    <input required type="email" name='email' class="form-control" placeholder="Email" >
    <div class="input-group-append">
      <div class="input-group-text">
        <span class="fas fa-envelope"></span>
      </div>
    </div>
  </div>
  <div class="input-group mb-3">
    <input required type="password" name='password' class="form-control"
placeholder="Password">
    <div class="input-group-append">
      <div class="input-group-text">
        <span class="fas fa-lock"></span>
      </div>
    </div>
  </div>
  <div class="input-group mb-3">
    <div class="g-recaptcha" data-
sitekey="6LfTGD4qAAAAJLKy1TGJY7uuTh_T1cB5VikRWjF"></div>

  </div>
  <div class="row">
    <div class="col-8">
      <div class="icheck-primary">
        <input type="checkbox" id="remember">
        <label for="remember">
          Remember Me
        </label>
      </div>
    </div>
    <!-- /.col -->
    <div class="col-4">
      <button type="submit" class="btn btn-success btn-block">Log In</button>
    </div>

    <!-- /.col -->

```

```

        </div>
    </form>

    <p class="mb-1">
        <a href="{% url 'password_reset' %}">Forgot Password?</a>
    </p>

</div>
<!-- /.login-card-body -->
</div>
</div>
<!-- /.login-box -->

<!-- jQuery -->
<script src="{% static 'plugins/jquery/jquery.min.js' %}"></script>
<!-- Bootstrap 4 -->
<script src="{% static 'plugins/bootstrap/js/bootstrap.bundle.min.js' %}"></script>
<!-- AdminLTE App -->
<script src="{% static 'dist/js/adminlte.min.js' %}"></script>

</body>

</html>

```

*****Base.html*****

```

{% load static %}
<!DOCTYPE html>
<html>

<head>
    <meta charset="utf-8">
    <meta http-equiv="X-UA-Compatible" content="IE=edge">
    <title>JECRC University</title>
    <!-- Tell the browser to be responsive to screen width -->
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <!-- Font Awesome -->
    <link rel="stylesheet" href="{% static 'plugins/fontawesome-free/css/all.min.css' %}">
    <!-- Ionicons -->
    <link rel="stylesheet"
href="https://code.ionicframework.com/ionicons/2.0.1/css/ionicons.min.css">
    <!-- favicon -->
    <link rel="shortcut icon" href="{% static '/image/favicon.ico' %}">
    <!-- Tempusdominus Bbootstrap 4 -->
    <link rel="stylesheet"
        href="{% static 'plugins/tempusdominus-bootstrap-4/css/tempusdominus-bootstrap-
4.min.css' %}">

```

```

<!-- iCheck -->
<link rel="stylesheet" href="{% static 'plugins/icheck-bootstrap/icheck-bootstrap.min.css'%} ">

<!-- Theme style -->
<link rel="stylesheet" href="{% static 'dist/css/adminlte.min.css'%} ">
<!-- overlayScrollbars -->
<link rel="stylesheet" href="{% static 'plugins/overlayScrollbars/css/OverlayScrollbars.min.css'%} ">
<!-- Daterange picker -->
<link rel="stylesheet" href="{% static 'plugins/daterangepicker/daterangepicker.css'%} ">

<!-- Google Font: Source Sans Pro -->
<link href="https://fonts.googleapis.com/css?family=Source+Sans+Pro:300,400,400i,700"
rel="stylesheet">
{% block custom_css %}

{% endblock custom_css %}
</head>

<body class="hold-transition sidebar-mini layout-fixed" >
  <div class="wrapper" >

    <!-- Navbar -->
    <nav class="main-header navbar navbar-expand navbar-light " style="background-color:
#343a40;">
      <!-- Left navbar links -->
      <ul class="navbar-nav">
        <li class="nav-item">
          <a class="nav-link" data-widget="pushmenu" href="#"><i style="color: white;" class="fas
fa-bars"></i></a>
        </li>

      </ul>

    </nav>
    <!-- /.navbar -->

    <!-- Main Sidebar Container -->

    {% include "main_app/sidebar_template.html" with user=user %}

    <div class="content-wrapper">

      <!-- Content Wrapper. Contains page content -->
      <!-- Content Header (Page header) -->
      <div class="content-header">
        <div class="container-fluid">
          <div class="row mb-2">

```

```

        <div class="col-sm-6">
            <h1 class="m-0 text-dark">{% block page_title %}{% endblock page_title %}</h1>
        </div><!-- /.col -->
        <div class="col-sm-6">

            <ol class="breadcrumb float-sm-right">
                <li class="breadcrumb-item"><a href="#">Home</a></li>
                <li class="breadcrumb-item active">{{ page_title }}</li>
            </ol>
        </div><!-- /.col -->
    </div><!-- /.row -->
</div><!-- /.container-fluid -->
</div>
<!-- Main content -->

<section class="content">
    <div class="container-fluid">
        <div class="row">
            <div class="col-md-12">
                <div class="form-group">
                    {% if messages %}
                    {% for message in messages %}

                        {% if message.tags == 'success' %}
                        <div class="alert alert-success">
                            {{message}}
                        </div>
                        {% else %}
                        <div class="alert alert-danger">
                            {{message}}
                        </div>
                        {% endif %}

                    {% endfor %}
                    {% endif %}
                </div>
            </div></div></div></section>
            {% block content %}

            {% endblock content %}
        <!-- /.content -->
    </div>
    <!-- /.content-wrapper -->
    {% include "main_app/footer.html" %}

</div>
<!-- ./wrapper -->

```

```

<!-- jQuery -->
<script src="{% static 'plugins/jquery/jquery.min.js'%}" "></script>
<!-- jQuery UI 1.11.4 -->
<script src="{% static 'plugins/jquery-ui/jquery-ui.min.js'%}" "></script>
<!-- Resolve conflict in jQuery UI tooltip with Bootstrap tooltip -->
<script>
    $.widget.bridge('uibutton', $.ui.button)
</script>
<!-- Bootstrap 4 -->
<script src="{% static 'plugins/bootstrap/js/bootstrap.bundle.min.js'%}" "></script>
<!-- ChartJS -->
<script src="{% static 'plugins/chart.js/Chart.min.js'%}" "></script>
<!-- Sparkline -->
<script src="{% static 'plugins/sparklines/sparkline.js'%}" "></script>

<!-- jQuery Knob Chart -->
<script src="{% static 'plugins/jquery-knob/jquery.knob.min.js'%}" "></script>
<!-- daterangepicker -->
<script src="{% static 'plugins/moment/moment.min.js'%}" "></script>
<script src="{% static 'plugins/daterangepicker/daterangepicker.js'%}" "></script>
<!-- Tempusdominus Bootstrap 4 -->
<script src="{% static 'plugins/tempusdominus-bootstrap-4/js/tempusdominus-bootstrap-4.min.js'%}" "></script>

<!-- overlayScrollbars -->
<script src="{% static 'plugins/overlayScrollbars/js/jquery.overlayScrollbars.min.js'%}" "></script>
<!-- AdminLTE App -->
<script src="{% static 'dist/js/adminlte.js'%}" "></script>
<!-- AdminLTE dashboard demo (This is only for demo purposes) -->
<script src="{% static 'dist/js/pages/dashboard.js'%}" "></script>
<!-- AdminLTE for demo purposes -->
<script src="{% static 'dist/js/demo.js'%}" "></script>
{% block custom_js %}

{% endblock custom_js %}
</body>

</html>

```

*****__Form-template.html__*****

```
{% load static %}
<!DOCTYPE html>
<html>

<head>
  <meta charset="utf-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <title>JECRC University</title>
  <!-- Tell the browser to be responsive to screen width -->
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <!-- Font Awesome -->
  <link rel="stylesheet" href="{% static 'plugins/fontawesome-free/css/all.min.css'%} ">
  <!-- Ionicons -->
  <link rel="stylesheet"
href="https://code.ionicframework.com/ionicons/2.0.1/css/ionicons.min.css">
  <!-- favicon -->
  <link rel="shortcut icon" href="{% static '/image/favicon.ico' %}">
  <!-- Tempusdominus Bbootstrap 4 -->
  <link rel="stylesheet"
    href="{% static 'plugins/tempusdominus-bootstrap-4/css/tempusdominus-bootstrap-
4.min.css'%} ">
  <!-- iCheck -->
  <link rel="stylesheet" href="{% static 'plugins/ichexk-bootstrap/ichexk-bootstrap.min.css'%} ">

  <!-- Theme style -->
  <link rel="stylesheet" href="{% static 'dist/css/adminlte.min.css'%} ">
  <!-- overlayScrollbars -->
  <link rel="stylesheet" href="{% static 'plugins/overlayScrollbars/css/OverlayScrollbars.min.css'%} ">
  <!-- Daterange picker -->
  <link rel="stylesheet" href="{% static 'plugins/daterangepicker/daterangepicker.css'%} ">

  <!-- Google Font: Source Sans Pro -->
  <link href="https://fonts.googleapis.com/css?family=Source+Sans+Pro:300,400,400i,700"
rel="stylesheet">
  {% block custom_css %}

  {% endblock custom_css %}
</head>

<body class="hold-transition sidebar-mini layout-fixed" >
  <div class="wrapper" >

    <!-- Navbar -->
    <nav class="main-header navbar navbar-expand navbar-light " style="background-color:
#343a40;">
      <!-- Left navbar links -->
      <ul class="navbar-nav">
```

```

        <li class="nav-item">
            <a class="nav-link" data-widget="pushmenu" href="#"><i style="color: white;" class="fas
fa-bars"></i></a>
        </li>

```

```

    </ul>

```

```

</nav>
<!-- /.navbar -->

```

```

<!-- Main Sidebar Container -->

```

```

{% include "main_app/sidebar_template.html" with user=user %}

```

```

<div class="content-wrapper">

```

```

    <!-- Content Wrapper. Contains page content -->

```

```

    <!-- Content Header (Page header) -->

```

```

    <div class="content-header">

```

```

        <div class="container-fluid">

```

```

            <div class="row mb-2">

```

```

                <div class="col-sm-6">

```

```

                    <h1 class="m-0 text-dark">{% block page_title %}{% endblock page_title %}</h1>

```

```

                </div><!-- /.col -->

```

```

                <div class="col-sm-6">

```

```

                    <ol class="breadcrumb float-sm-right">

```

```

                        <li class="breadcrumb-item"><a href="#">Home</a></li>

```

```

                        <li class="breadcrumb-item active">{{ page_title }}</li>

```

```

                    </ol>

```

```

                </div><!-- /.col -->

```

```

            </div><!-- /.row -->

```

```

        </div><!-- /.container-fluid -->

```

```

    </div>

```

```

    <!-- /.content-header -->

```

```

    <!-- Main content -->

```

```

<section class="content">

```

```

    <div class="container-fluid">

```

```

        <div class="row">

```

```

            <div class="col-md-12">

```

```

                <div class="form-group">

```

```

                    {% if messages %}

```

```

                    {% for message in messages %}

```

```

                        {% if message.tags == 'success' %}

```

```

        <div class="alert alert-success">
            {{message}}
        </div>
        {% else %}
        <div class="alert alert-danger">
            {{message}}
        </div>
        {% endif %}

        {% endfor %}
        {% endif %}
    </div>
</div></div></div></section>
{% block content %}

    {% endblock content %}
<!-- /.content -->
</div>
<!-- /.content-wrapper -->
{% include "main_app/footer.html" %}

</div>
<!-- ./wrapper -->
<!-- jQuery -->
<script src="{% static 'plugins/jquery/jquery.min.js'%}" "></script>
<!-- jQuery UI 1.11.4 -->
<script src="{% static 'plugins/jquery-ui/jquery-ui.min.js'%}" "></script>
<!-- Resolve conflict in jQuery UI tooltip with Bootstrap tooltip -->
<script>
    $.widget.bridge('uibutton', $.ui.button)
</script>
<!-- Bootstrap 4 -->
<script src="{% static 'plugins/bootstrap/js/bootstrap.bundle.min.js'%}" "></script>
<!-- ChartJS -->
<script src="{% static 'plugins/chart.js/Chart.min.js'%}" "></script>
<!-- Sparkline -->
<script src="{% static 'plugins/sparklines/sparkline.js'%}" "></script>

<!-- jQuery Knob Chart -->
<script src="{% static 'plugins/jquery-knob/jquery.knob.min.js'%}" "></script>
<!-- daterangepicker -->
<script src="{% static 'plugins/moment/moment.min.js'%}" "></script>
<script src="{% static 'plugins/daterangepicker/daterangepicker.js'%}" "></script>
<!-- Tempusdominus Bootstrap 4 -->
<script src="{% static 'plugins/tempusdominus-bootstrap-4/js/tempusdominus-bootstrap-4.min.js'%}" "></script>

<!-- overlayScrollbars -->
<script src="{% static 'plugins/overlayScrollbars/js/jquery.overlayScrollbars.min.js'%}" "></script>

```

```

<!-- AdminLTE App -->
<script src="{% static 'dist/js/adminlte.js'%} "></script>
<!-- AdminLTE dashboard demo (This is only for demo purposes) -->
<script src="{% static 'dist/js/pages/dashboard.js'%} "></script>
<!-- AdminLTE for demo purposes -->
<script src="{% static 'dist/js/demo.js'%} "></script>
{% block custom_js %}

{% endblock custom_js %}
</body>

</html>

```

*****__Reset-password.html__*****

```

{% load static %}
<!DOCTYPE html>
<html>

<head>
  <meta charset="utf-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <title>C.M.S | Log In</title>
  <!-- Tell the browser to be responsive to screen width -->
  <meta name="viewport" content="width=device-width, initial-scale=1">

  <!-- Font Awesome -->
  <link rel="stylesheet" href="{% static 'plugins/fontawesome-free/css/all.min.css' %}">
  <!-- Ionicons -->
  <link rel="stylesheet" href="{% static
'https://code.ionicframework.com/ionicons/2.0.1/css/ionicons.min.css' %}">
  <!-- icheck bootstrap -->
  <link rel="stylesheet" href="{% static 'plugins/icheck-bootstrap/icheck-bootstrap.min.css' %}">
  <!-- Theme style -->
  <link rel="stylesheet" href="{% static 'dist/css/adminlte.min.css' %}">
  <!-- Google Font: Source Sans Pro -->
  <link href="https://fonts.googleapis.com/css?family=Source+Sans+Pro:300,400,400i,700"
rel="stylesheet">
</head>

<body class="hold-transition login-page">
  <div class="login-box">
    <div class="login-logo">
      <a href=""><b>C.M.S</b></a>

    </div>
    <!-- /.login-logo -->
    <div class="card">

```

```

        {% block content %}

        {% endblock content %}
    </div>
</div>
<!-- /.login-box -->

<!-- jQuery -->
<script src="{% static 'plugins/jquery/jquery.min.js' %}"></script>
<!-- Bootstrap 4 -->
<script src="{% static 'plugins/bootstrap/js/bootstrap.bundle.min.js' %}"></script>
<!-- AdminLTE App -->
<script src="{% static 'dist/js/adminlte.min.js' %}"></script>

</body>

</html>

*****_____Home.html (Staff)_____*****

```

```

{% extends 'main_app/base.html' %}
{% load static %}
{% block page_title %}{{page_title}}{% endblock page_title %}
{% block content %}
<section class="content">
    <div class="container-fluid">
        <!-- Small boxes (Stat box) -->
        <div class="row">
            <div class="col-lg-3 col-6">
                <!-- small box -->
                <div class="small-box bg-info">
                    <div class="inner">
                        <h3>{{total_students}}</h3>

                        <p>Total Students</p>
                    </div>
                    <div class="icon">
                        <i class="nav-icon fas fa-user-graduate"></i>
                    </div>

                </div>
            </div>
            <!-- ./col -->
            <div class="col-lg-3 col-6">
                <!-- small box -->
                <div class="small-box bg-success">
                    <div class="inner">
                        <h3>{{total_attendance}}</h3>

```

```

        <p>Total Attendance Taken</p>
    </div>
    <div class="icon">
        <i class="nav-icon fas fa-calendar-alt"></i>
    </div>

</div>
</div>
<!-- ./col -->
<div class="col-lg-3 col-6">
    <!-- small box -->
    <div class="small-box bg-maroon">
        <div class="inner">
            <h3>{{total_leave}}</h3>

            <p>Total Leave Applied</p>
        </div>
        <div class="icon">
            <i class="nav-icon fas fa-sign-out-alt"></i>
        </div>

    </div>
</div>
<!-- ./col -->
<div class="col-lg-3 col-6">
    <!-- small box -->
    <div class="small-box bg-danger">
        <div class="inner">
            <h3>{{total_subject}}</h3>

            <p>Total Subjects</p>
        </div>
        <div class="icon">
            <i class="nav-icon fas fa-book"></i>
        </div>

    </div>
</div>
<!-- ./col -->
</div>
<!-- /row -->
<!-- Main row -->
<div class="row">
    <div class="col-md-6">
        <!-- LINE CHART -->
        <div class="card card-secondary">
            <div class="card-header">
                <h3 class="card-title">{{page_title}}</h3>

```

```

        <div class="card-tools">
            <button type="button" class="btn btn-tool" data-card-widget="collapse"><i class="fas fa-
minus"></i>
            </button>
            <button type="button" class="btn btn-tool" data-card-widget="remove"><i class="fas fa-
times"></i></button>
        </div>
    </div>
    <div class="card-body">
        <div class="chart">
            <canvas id="pieChart" style="min-height: 250px; height: 250px; max-height: 250px; max-
width: 100%;"></canvas>
        </div>
    </div></div>
    <!-- /.card-body -->
</div>
<div class="col-md-6">
    <div class="card card-secondary">
<div class="card-header">
    <h3 class="card-title">{{page_title}}</h3>

    <div class="card-tools">
        <button type="button" class="btn btn-tool" data-card-widget="collapse"><i class="fas fa-
minus"></i>
        </button>
        <button type="button" class="btn btn-tool" data-card-widget="remove"><i class="fas fa-
times"></i></button>
    </div>
</div>
    <div class="card-body">
        <div class="chart">
            <canvas id="barChart" style="min-height: 250px; height: 250px; max-height: 250px; max-
width: 100%;"></canvas>
        </div>
    </div>
    <!-- /.card-body -->
</div>
<!-- right col -->
</div>
<!-- /.row (main row) -->
</div><!-- /.container-fluid -->
</section>
{% endblock content %}

{% block custom_js %}
<script>
    $(document).ready(function(){

```

```

var donutData = {
  labels: ['Attendance', 'Leave'],
  datasets: [
    {
      data: [{total_attendance}}, {total_leave}}],
      backgroundColor : ['#00a65a', '#f39c12'],
    }
  ]
}
var pieChartCanvas = $('#pieChart').get(0).getContext('2d')
var pieData = donutData;
var pieOptions = {
  maintainAspectRatio : false,
  responsive : true,
}
//Create pie or douhnut chart
// You can switch between pie and douhnut using the method below.
var pieChart = new Chart(pieChartCanvas, {
  type: 'pie',
  data: pieData,
  options: pieOptions
});

var subject_list = {{ subject_list|safe|escape }};
var attendance_list = {{ attendance_list }};

var barChartData = {
labels : subject_list,
datasets: [
{
  label : 'Attendance Per Subject',
  backgroundColor : '#17A2B8',
  borderColor : 'rgba(60,141,188,0.8)',
  pointRadius : false,
  pointColor : '#3b8bba',
  pointStrokeColor : 'rgba(60,141,188,1)',
  pointHighlightFill : '#fff',
  pointHighlightStroke: 'rgba(60,141,188,1)',
  data : attendance_list
},
]
}
var barChartCanvas = $('#barChart').get(0).getContext('2d')
var temp0 = barChartData.datasets[0]
//var temp1 = areaChartData.datasets[1]
barChartData.datasets[0] = temp0
// barChartData.datasets[1] = temp0

```

```

var stackedBarChartOptions = {
  responsive      : true,
  maintainAspectRatio : false,
  scales: {
    xAxes: [{
      stacked: true,
    }],
    yAxes: [{
      stacked: true
    }]
  }
}

var barChart = new Chart(barChartCanvas, {
  type: 'bar',
  data: barChartData,
  options: stackedBarChartOptions
})

```

</script>

<!-- The core Firebase JS SDK is always required and must be listed first -->

<script src="https://www.gstatic.com/firebasejs/7.23.0/firebase-app.js"></script>

<!-- TODO: Add SDKs for Firebase products that you want to use

<https://firebase.google.com/docs/web/setup#available-libraries> -->

<script src="https://www.gstatic.com/firebasejs/7.23.0/firebase-analytics.js"></script>

<script src="https://www.gstatic.com/firebasejs/7.22.1/firebase-messaging.js"></script>

<script>

// Your web app's Firebase configuration

// For Firebase JS SDK v7.20.0 and later, measurementId is optional

```

var firebaseConfig = {
  apiKey: "AlzaSyBarDWWHTfTMSrtc5Lj3Cdw5dEvjAkFwtM",
  authDomain: "sms-with-django.firebaseio.com",
  databaseURL: "https://sms-with-django.firebaseio.com",
  projectId: "sms-with-django",
  storageBucket: "sms-with-django.appspot.com",
  messagingSenderId: "945324593139",
  appId: "1:945324593139:web:03fa99a8854bbd38420c86",
  measurementId: "G-2F2RXTL9GT"
};

```

// Initialize Firebase

firebase.initializeApp(firebaseConfig)

const messaging = firebase.messaging();

function InitializeFireBaseMessaging() {

 messaging

```

        .requestPermission()
        .then(function () {
            console.log("Notification Permission");
            return messaging.getToken();
        })
        .then(function (token) {
            console.log("Token : " + token);
            sendToServer(token);
        })
        .catch(function (reason) {
            console.log(reason)
        })
    }
    messaging.onMessage(function (payload) {
        const notificationOption = {
            body: payload.notification.body,
            icon: payload.notification.icon,

        }
        if (Notification.permission == 'granted') {
            var notification = new Notification(payload.notification.title, notificationOption);
            notification.onclick = function (event) {
                event.preventDefault();
                window.open(payload.notification.click_action, "_blank");
                notification.close();
            }
        }
        console.log(payload);
    });
    messaging.onTokenRefresh(function () {
        messaging.getToken()
        .then(function (newToken) {
            console.log("New Token : " + newToken);
            sendToServer(newToken);

        })
        .catch(function (reason) {
            console.log(reason)
        })
    })

    function sendToServer(token){
        $.ajax({
            url: "{% url 'staff_fcmtoken' %}",
            type: 'POST',
            data: {
                token: token,
            }
        }).done(function (response) {

```

```

    }).fail(function (response) {
    })

}

InitializeFirebaseMessaging();
</script>
{% endblock custom_js %}

```

*****__Home.html (Student)_____*****

```

{% extends 'main_app/base.html' %}
{% load static %}
{% block page_title %}{{page_title}}{% endblock page_title %}
{% block content %}
<section class="content">
  <div class="container-fluid">
    <!-- Small boxes (Stat box) -->
    <div class="row">
      <div class="col-lg-3 col-6">
        <!-- small box -->
        <div class="small-box bg-info">
          <div class="inner">
            <h3>{{total_attendance}}</h3>

            <p>Total Attendance</p>
          </div>
          <div class="icon">
            <i class="nav-icon fas fa-calendar-alt"></i>
          </div>
          <# <a href="#" class="small-box-footer">More info <i class="fas fa-arrow-circle-
right"></i></a> #>
        </div>
      </div>
      <!-- ./col -->
      <div class="col-lg-3 col-6">
        <!-- small box -->
        <div class="small-box bg-success">
          <div class="inner">
            <h3>{{ percent_present }}<sup style="font-size: 20px">%</sup></h3>

            <p>Percentage Present</p>
          </div>
          <div class="icon">
            <i class="nav-icon fas fa-calendar-check"></i>
          </div>

```

```

        {# <a href="#" class="small-box-footer">More info <i class="fas fa-arrow-circle-
right"></i></a> #}
    </div>
</div>
<!-- ./col -->
<div class="col-lg-3 col-6">
    <!-- small box -->
    <div class="small-box bg-danger">
        <div class="inner">
            <h3>{{percent_absent}}<sup style="font-size: 20px">%</sup></h3>

            <p>Percentage Absent</p>
        </div>
        <div class="icon">
            <i class="nav-icon fas fa-calendar-minus"></i>
        </div>
        {# <a href="#" class="small-box-footer">More info <i class="fas fa-arrow-circle-
right"></i></a> #}
    </div>
</div>
<!-- ./col -->
<div class="col-lg-3 col-6">
    <!-- small box -->
    <div class="small-box bg-primary">
        <div class="inner">
            <h3>{{total_subject}}</h3>

            <p>Total Subject</p>
        </div>
        <div class="icon">
            <i class="nav-icon fas fa-book"></i>
        </div>
        {# <a href="#" class="small-box-footer">More info <i class="fas fa-arrow-circle-
right"></i></a> #}
    </div>
</div>
<!-- ./col -->
</div>
<div class="row">
    <div class="col-lg-6">

        <!-- DONUT CHART -->
        <div class="card card-secondary">
            <div class="card-header">
                <h3 class="card-title">{{page_title}}</h3>

                <div class="card-tools">
                    <button type="button" class="btn btn-tool" data-card-widget="collapse"><i
class="fas fa-minus"></i>

```



```

var data_present = {{data_present}}
var data_absent = {{data_absent}}

//-----
// - DONUT CHART -
var attendanceDataCanvas = $('#attendanceData').get(0).getContext('2d')
var donutData = {
  labels: [
    'Present', 'Absent'
  ],
  datasets: [
    {
      data: [{percent_present}, {percent_absent}],
      backgroundColor : ['#00a65a', '#f56954'],
    }
  ]
}
var donutOptions = {
  maintainAspectRatio : false,
  responsive : true,
}
var attendanceData = new Chart(attendanceDataCanvas, {
  type: 'doughnut',
  data: donutData,
  options: donutOptions
});

//attendanceStatistics
//Bar Chart
var areaChartData = {
  labels : subjects,
  datasets: [
    {
      label : 'Present In Class',
      backgroundColor : 'rgba(60,141,188,0.9)',
      borderColor : 'rgba(60,141,188,0.8)',
      pointRadius : false,
      pointColor : '#3b8bba',
      pointStrokeColor : 'rgba(60,141,188,1)',
      pointHighlightFill : '#fff',
      pointHighlightStroke: 'rgba(60,141,188,1)',
      data : data_present
    },
    {
      label : 'Absent In Class',
      backgroundColor : 'rgba(210, 214, 222, 1)',
      borderColor : 'rgba(210, 214, 222, 1)',
      pointRadius : false,

```

```

        pointColor      : 'rgba(210, 214, 222, 1)',
        pointStrokeColor : '#c1c7d1',
        pointHighlightFill : '#fff',
        pointHighlightStroke: 'rgba(220,220,220,1)',
        data             : data_absent
    },
]
}
var barChartCanvas = $('#attendanceStatistics').get(0).getContext('2d')
var barChartData = jQuery.extend(true, {}, areaChartData)
var temp = areaChartData.datasets[0]
barChartData.datasets[0] = temp
var barChartOptions = {
    responsive          : true,
    maintainAspectRatio : false,
    datasetFill         : false
}

var barChart = new Chart(barChartCanvas, {
    type: 'bar',
    data: barChartData,
    options: barChartOptions
})

})

</script>
<!-- The core Firebase JS SDK is always required and must be listed first -->
<script src="https://www.gstatic.com/firebasejs/7.23.0/firebase-app.js"></script>

<!-- TODO: Add SDKs for Firebase products that you want to use
https://firebase.google.com/docs/web/setup#available-libraries -->
<script src="https://www.gstatic.com/firebasejs/7.23.0/firebase-analytics.js"></script>
<script src="https://www.gstatic.com/firebasejs/7.22.1/firebase-messaging.js"></script>

<script>
    // Your web app's Firebase configuration
    // For Firebase JS SDK v7.20.0 and later, measurementId is optional
    var firebaseConfig = {
        apiKey: "AlzaSyBarDWWHTfTMSrtc5Lj3Cdw5dEvjAkFwtM",
        authDomain: "sms-with-django.firebaseio.com",
        databaseURL: "https://sms-with-django.firebaseio.com",
        projectId: "sms-with-django",
        storageBucket: "sms-with-django.appspot.com",
        messagingSenderId: "945324593139",
        appId: "1:945324593139:web:03fa99a8854bbd38420c86",
        measurementId: "G-2F2RXTL9GT"
    }

```

```

};
// Initialize Firebase
firebase.initializeApp(firebaseConfig)
const messaging = firebase.messaging();
function InitializeFireBaseMessaging() {
  messaging
    .requestPermission()
    .then(function () {
      console.log("Notification Permission");
      return messaging.getToken();
    })
    .then(function (token) {
      console.log("Token : " + token);
      sendToServer(token);
    })
    .catch(function (reason) {
      console.log(reason)
    })
}
messaging.onMessage(function (payload) {
  const notificationOption = {
    body: payload.notification.body,
    icon: payload.notification.icon,

  }
  if (Notification.permission === 'granted') {
    var notification = new Notification(payload.notification.title, notificationOption);
    notification.onclick = function (event) {
      event.preventDefault();
      window.open(payload.notification.click_action, "_blank");
      notification.close();
    }
  }
  console.log(payload);
});
messaging.onTokenRefresh(function () {
  messaging.getToken()
    .then(function (newToken) {
      console.log("New Token : " + newToken);
      sendToServer(newToken);

    })
    .catch(function (reason) {
      console.log(reason)
    })
})

function sendToServer(token){
  $.ajax({

```

```

        url: "{% url 'student_fcmtoken' %}",
        type: 'POST',
        data: {
            token: token,
        }
    }).done(function (response) {

    })

    }).fail(function (response) {
    })

}

InitializeFireBaseMessaging();
</script>
{% endblock custom_js %}

```

***** Home.html (Admin) *****

```

{% extends 'main_app/base.html' %}
{% load static %}
{% block page_title %}{{page_title}}{% endblock page_title %}
{% block content %}
<section class="content">
    <div class="container-fluid">
        <!-- Small boxes (Stat box) -->
        <div class="row">
            <div class="col-lg-3 col-6">
                <!-- small box -->
                <div class="small-box bg-info">
                    <div class="inner">
                        <h3>{{total_students}}</h3>

                        <p>Total Students</p>
                    </div>
                    <div class="icon">
                        <i class="nav-icon fas fa-user-graduate"></i>
                    </div>
                    <a href="{% url 'manage_student' %}" class="small-box-footer">More info <i class="fas fa-arrow-circle-right"></i></a>

                </div>
            </div>
            <!-- ./col -->
            <div class="col-lg-3 col-6">
                <!-- small box -->
                <div class="small-box bg-success">

```

```

        <div class="inner">
            <h3>{{total_staff}}</h3>

            <p>Total Staff</p>
        </div>
        <div class="icon">
            <i class="nav-icon fas fa-users"></i>
        </div>
        <a href="{% url 'manage_staff' %}" class="small-box-footer">More info <i class="fas fa-
arrow-circle-right"></i></a>
    </div>
</div>
<!-- ./col -->
<div class="col-lg-3 col-6">
    <!-- small box -->
    <div class="small-box bg-purple">
        <div class="inner">
            <h3>{{total_course}}</h3>

            <p>Total Course</p>
        </div>
        <div class="icon">
            <i class="nav-icon fas fa-th-list"></i>
        </div>
        <a href="{% url 'manage_course' %}" class="small-box-footer">More info <i class="fas fa-
arrow-circle-right"></i></a>
    </div>
</div>
<!-- ./col -->
<div class="col-lg-3 col-6">
    <!-- small box -->
    <div class="small-box bg-danger">
        <div class="inner">
            <h3>{{total_subject}}</h3>

            <p>Total Subjects</p>
        </div>
        <div class="icon">
            <i class="nav-icon fas fa-book"></i>
        </div>
        <a href="{% url 'manage_subject' %}" class="small-box-footer">More info <i class="fas fa-
arrow-circle-right"></i></a>
    </div>
</div>
<!-- ./col -->
</div>
<!-- Main row -->
<div class="row">

```

```

<div class="col-md-6">
  <!-- LINE CHART -->
  <div class="card card-dark">
    <div class="card-header">
      <h3 class="card-title">Staffs - Students Overview</h3>

      <div class="card-tools">
        <button type="button" class="btn btn-tool" data-card-widget="collapse"><i class="fas fa-
minus"></i>
          </button>
        <button type="button" class="btn btn-tool" data-card-widget="remove"><i class="fas fa-
times"></i></button>
      </div>
    </div>
    <div class="card-body">
      <div class="chart">
        <canvas id="pieChart" style="min-height: 250px; height: 250px; max-height: 250px; max-
width: 100%;"></canvas>
      </div>
    </div></div>
    <!-- /.card-body -->
  </div>
  <div class="col-md-6">
    <div class="card card-dark">
      <div class="card-header">
        <h3 class="card-title">Attendance Per Subject</h3>

        <div class="card-tools">
          <button type="button" class="btn btn-tool" data-card-widget="collapse"><i class="fas fa-
minus"></i>
            </button>
          <button type="button" class="btn btn-tool" data-card-widget="remove"><i class="fas fa-
times"></i></button>
        </div>
      </div>
      <div class="card-body">
        <div class="chart">
          <canvas id="barChart" style="min-height: 250px; height: 250px; max-height: 250px; max-
width: 100%;"></canvas>
        </div>
      </div>
    <!-- /.card-body -->
  </div>
  <!-- right col -->
</div>
<!-- /.row (main row) -->
<div class="row">
  <div class="col-lg-12">

```

```

<!-- BAR CHART -->
<div class="card card-dark">
  <div class="card-header">
    <h3 class="card-title">Student's Attendance & Leave Overview</h3>

    <div class="card-tools">
      <button type="button" class="btn btn-tool" data-card-widget="collapse"><i class="fas fa-minus"></i>
    </button>
      <button type="button" class="btn btn-tool" data-card-widget="remove"><i class="fas fa-times"></i></button>
    </div>
  </div>
  <div class="card-body">
    <div class="chart">
      <canvas id="barChart2" style="min-height: 250px; height: 250px; max-height: 250px; max-width: 100%;"></canvas>
    </div>
  </div>
<!-- /.card-body -->
</div>
<!-- /.card -->
</div>

<div class="col-lg-6">

</div>

</div>

<div class="row">

<div class="col-lg-6">
  <!-- PIE CHART -->
  <div class="card card-dark">
    <div class="card-header">
      <h3 class="card-title">Total Students in Each Course</h3>

      <div class="card-tools">
        <button type="button" class="btn btn-tool" data-card-widget="collapse"><i class="fas fa-minus"></i>
        </button>
        <button type="button" class="btn btn-tool" data-card-widget="remove"><i class="fas fa-times"></i></button>
      </div>
    </div>
    <div class="card-body">
      <canvas id="pieChart2" style="min-height: 250px; height: 250px; max-height: 250px; max-width: 100%;"></canvas>
    </div>
  </div>
</div>

```

```

    </div>
    <!-- /.card-body -->
    </div>
</div>

<div class="col-lg-6">
    <!-- PIE CHART -->
    <div class="card card-dark">
        <div class="card-header">
            <h3 class="card-title">Total Students in Each Subject</h3>

            <div class="card-tools">
                <button type="button" class="btn btn-tool" data-card-widget="collapse"><i class="fas fa-minus"></i>
                </button>
                <button type="button" class="btn btn-tool" data-card-widget="remove"><i class="fas fa-times"></i></button>
            </div>
        </div>
        <div class="card-body">
            <canvas id="pieChart3" style="min-height: 250px; height: 250px; max-height: 250px; max-width: 100%;"></canvas>
        </div>
    <!-- /.card-body -->
    </div>
</div>
</div><!-- /.container-fluid -->
</section>
{% endblock content %}

{% block custom_js %}
<script>
    $(document).ready(function(){
        var donutData = {
            labels: ['Students', 'Staff'],
            datasets: [
                {
                    data:[{{total_students}}, {{total_staff}}],
                    backgroundColor : ['#00a65a', '#f39c12'],
                }
            ]
        }
        var pieChartCanvas = $('#pieChart').get(0).getContext('2d')
        var pieData = donutData;
        var pieOptions = {
            maintainAspectRatio : false,
            responsive : true,
        }
    }

```

```

//Create pie or douhnut chart
// You can switch between pie and douhnut using the method below.
var pieChart = new Chart(pieChartCanvas, {
  type: 'pie',
  data: pieData,
  options: pieOptions
});

var subject_list = {{ subject_list|safe|escape }};
var attendance_list = {{ attendance_list }};

var barChartData = {
labels : subject_list,
datasets: [
{
  label      : 'Attendance Per Subject',
  backgroundColor  : '#17A2B8',
  borderColor      : 'rgba(60,141,188,0.8)',
  pointRadius      : false,
  pointColor       : '#3b8bba',
  pointStrokeColor : 'rgba(60,141,188,1)',
  pointHighlightFill : '#fff',
  pointHighlightStroke: 'rgba(60,141,188,1)',
  data            : attendance_list
},
]
}

var barChartCanvas = $('#barChart').get(0).getContext('2d')
var temp0 = barChartData.datasets[0]
//var temp1 = areaChartData.datasets[1]
barChartData.datasets[0] = temp0
// barChartData.datasets[1] = temp0

var stackedBarChartOptions = {
  responsive      : true,
  maintainAspectRatio  : false,
  scales: {
    xAxes: [{
      stacked: true,
    }],
    yAxes: [{
      stacked: true
    }]
  }
}

var barChart = new Chart(barChartCanvas, {
  type: 'bar',

```

```

        data: barChartData,
        options: stackedBarChartOptions
    })

    // Total Students in Each Subject
    var student_count_list_in_subject = {{ student_count_list_in_subject }}
    var subject_list = {{ subject_list|safe }}
    var pieData3 = {
        labels: subject_list,
        datasets: [
            {
                data: student_count_list_in_subject,
                backgroundColor : ['#8f251f', '#00a65a', '#f39c12', '#00c0ef', '#3c8dbc', '#894e9c', '#0b523a',
'#a1156d', '#9e4603'],
            }
        ]
    }

    //-----
    //- PIE CHART -
    //-----
    // Get context with jQuery - using jQuery's .get() method.
    var pieChartCanvas3 = $('#pieChart3').get(0).getContext('2d')
    var pieData3      = pieData3;
    var pieOptions3   = {
        maintainAspectRatio : false,
        responsive : true,
    }

    var pieChart3 = new Chart(pieChartCanvas3, {
        type: 'pie',
        data: pieData3,
        options: pieOptions3
    })

    // - BAR CHART - Student Attendance & Leave Overview //
    var student_attendance_present_list = {{ student_attendance_present_list }};
    var student_attendance_leave_list = {{ student_attendance_leave_list }};
    var student_name_list = {{ student_name_list|safe }};

    var areaChartData2 = {
        labels : student_name_list,
        datasets: [
            {
                label      : 'Absent/Leave',
                backgroundColor : '#b50a04',
            }
        ]
    }

```

```

        borderColor      : '#b50a04',
        pointRadius      : false,
        pointColor       : '#3b8bba',
        pointStrokeColor : 'rgba(60,141,188,1)',
        pointHighlightFill : '#fff',
        pointHighlightStroke: 'rgba(60,141,188,1)',
        data              : student_attendance_leave_list
    },
    {
        label            : 'Attendance',
        backgroundColor   : '#5a615c',
        borderColor      : 'rgba(210, 214, 222, 1)',
        pointRadius      : false,
        pointColor       : 'rgba(210, 214, 222, 1)',
        pointStrokeColor : '#c1c7d1',
        pointHighlightFill : '#fff',
        pointHighlightStroke: 'rgba(220,220,220,1)',
        data              : student_attendance_present_list
    },
]
}

```

```

var barChartCanvas2 = $('#barChart2').get(0).getContext('2d')
var barChartData2 = jQuery.extend(true, {}, areaChartData2)
var temp02 = areaChartData2.datasets[0]
var temp12 = areaChartData2.datasets[1]
barChartData2.datasets[0] = temp12
barChartData2.datasets[1] = temp02

```

```

var barChartOptions2 = {
    responsive          : true,
    maintainAspectRatio : false,
    datasetFill         : false
}

```

```

var barChart2 = new Chart(barChartCanvas2, {
    type: 'bar',
    data: barChartData2,
    options: barChartOptions2
})

```

```

// Total Students in Each Course
//var donutChartCanvas = $('#pieChart2').get(0).getContext('2d')
var course_name_list = {{ course_name_list|safe }}
var student_count_list_in_course = {{ student_count_list_in_course }}
var pieData2 = {
    labels: course_name_list,
    datasets: [

```

```

        {
            data: student_count_list_in_course,
            backgroundColor : ['#cc0404', '#00a65a', '#f39c12', '#00A4BD', '#045c8f', '#894e9c', '#9e4603',
'#71bfb2'],
        }
    ]
}

//-----
// - PIE CHART -
//-----
// Get context with jQuery - using jQuery's .get() method.
var pieChartCanvas2 = $('#pieChart2').get(0).getContext('2d')
var pieData2      = pieData2;
var pieOptions2   = {
    maintainAspectRatio : false,
    responsive : true,
}

var pieChart2 = new Chart(pieChartCanvas2, {
    type: 'pie',
    data: pieData2,
    options: pieOptions2
})

})
</script>
{% endblock custom_js %}

```

***** **hod-views.py** *****

```

import json
import requests
from django.contrib import messages
from django.core.files.storage import FileSystemStorage
from django.http import HttpResponse, JsonResponse
from django.shortcuts import (HttpResponse, HttpResponseRedirect,
                             get_object_or_404, redirect, render)
from django.template.tags.static import static
from django.urls import reverse
from django.views.decorators.csrf import csrf_exempt
from django.views.generic import UpdateView

from .forms import *
from .models import *

```

```

def admin_home(request):
    total_staff = Staff.objects.all().count()
    total_students = Student.objects.all().count()
    subjects = Subject.objects.all()
    total_subject = subjects.count()
    total_course = Course.objects.all().count()
    attendance_list = Attendance.objects.filter(subject__in=subjects)
    total_attendance = attendance_list.count()
    attendance_list = []
    subject_list = []
    for subject in subjects:
        attendance_count = Attendance.objects.filter(subject=subject).count()
        subject_list.append(subject.name[:7])
        attendance_list.append(attendance_count)

    # Total Subjects and students in Each Course
    course_all = Course.objects.all()
    course_name_list = []
    subject_count_list = []
    student_count_list_in_course = []

    for course in course_all:
        subjects = Subject.objects.filter(course_id=course.id).count()
        students = Student.objects.filter(course_id=course.id).count()
        course_name_list.append(course.name)
        subject_count_list.append(subjects)
        student_count_list_in_course.append(students)

    subject_all = Subject.objects.all()
    subject_list = []
    student_count_list_in_subject = []
    for subject in subject_all:
        course = Course.objects.get(id=subject.course.id)
        student_count = Student.objects.filter(course_id=course.id).count()
        subject_list.append(subject.name)
        student_count_list_in_subject.append(student_count)

    # For Students
    student_attendance_present_list=[]
    student_attendance_leave_list=[]
    student_name_list=[]

    students = Student.objects.all()
    for student in students:

        attendance = AttendanceReport.objects.filter(student_id=student.id, status=True).count()
        absent = AttendanceReport.objects.filter(student_id=student.id, status=False).count()
        leave = LeaveReportStudent.objects.filter(student_id=student.id, status=1).count()

```

```

student_attendance_present_list.append(attendance)
student_attendance_leave_list.append(leave+absent)
student_name_list.append(student.admin.first_name)

```

```

context = {
    'page_title': "Administrative Dashboard",
    'total_students': total_students,
    'total_staff': total_staff,
    'total_course': total_course,
    'total_subject': total_subject,
    'subject_list': subject_list,
    'attendance_list': attendance_list,
    'student_attendance_present_list': student_attendance_present_list,
    'student_attendance_leave_list': student_attendance_leave_list,
    "student_name_list": student_name_list,
    "student_count_list_in_subject": student_count_list_in_subject,
    "student_count_list_in_course": student_count_list_in_course,
    "course_name_list": course_name_list,
}

return render(request, 'hod_template/home_content.html', context)

```

```

def add_staff(request):
    form = StaffForm(request.POST or None, request.FILES or None)
    context = {'form': form, 'page_title': 'Add Staff'}
    if request.method == 'POST':
        if form.is_valid():
            first_name = form.cleaned_data.get('first_name')
            last_name = form.cleaned_data.get('last_name')
            address = form.cleaned_data.get('address')
            email = form.cleaned_data.get('email')
            gender = form.cleaned_data.get('gender')
            password = form.cleaned_data.get('password')
            course = form.cleaned_data.get('course')
            passport = request.FILES.get('profile_pic')
            fs = FileSystemStorage()
            filename = fs.save(passport.name, passport)
            passport_url = fs.url(filename)
            try:
                user = CustomUser.objects.create_user(
                    email=email, password=password, user_type=2, first_name=first_name,
last_name=last_name, profile_pic=passport_url)
                user.gender = gender
                user.address = address
                user.staff.course = course
                user.save()
                messages.success(request, "Successfully Added")
                return redirect(reverse('add_staff'))
            except:

```

```

        except Exception as e:
            messages.error(request, "Could Not Add " + str(e))
        else:
            messages.error(request, "Please fulfil all requirements")

    return render(request, 'hod_template/add_staff_template.html', context)

def add_student(request):
    student_form = StudentForm(request.POST or None, request.FILES or None)
    context = {'form': student_form, 'page_title': 'Add Student'}
    if request.method == 'POST':
        if student_form.is_valid():
            first_name = student_form.cleaned_data.get('first_name')
            last_name = student_form.cleaned_data.get('last_name')
            address = student_form.cleaned_data.get('address')
            email = student_form.cleaned_data.get('email')
            gender = student_form.cleaned_data.get('gender')
            password = student_form.cleaned_data.get('password')
            course = student_form.cleaned_data.get('course')
            session = student_form.cleaned_data.get('session')
            passport = request.FILES['profile_pic']
            fs = FileSystemStorage()
            filename = fs.save(passport.name, passport)
            passport_url = fs.url(filename)
            try:
                user = CustomUser.objects.create_user(
                    email=email, password=password, user_type=3, first_name=first_name,
last_name=last_name, profile_pic=passport_url)
                user.gender = gender
                user.address = address
                user.student.session = session
                user.student.course = course
                user.save()
                messages.success(request, "Successfully Added")
                return redirect(reverse('add_student'))
            except Exception as e:
                messages.error(request, "Could Not Add: " + str(e))
            else:
                messages.error(request, "Could Not Add: ")
    return render(request, 'hod_template/add_student_template.html', context)

def add_course(request):
    form = CourseForm(request.POST or None)
    context = {
        'form': form,
        'page_title': 'Add Course'
    }

```

```

    }
    if request.method == 'POST':
        if form.is_valid():
            name = form.cleaned_data.get('name')
            try:
                course = Course()
                course.name = name
                course.save()
                messages.success(request, "Successfully Added")
                return redirect(reverse('add_course'))
            except:
                messages.error(request, "Could Not Add")
        else:
            messages.error(request, "Could Not Add")
    return render(request, 'hod_template/add_course_template.html', context)

def add_subject(request):
    form = SubjectForm(request.POST or None)
    context = {
        'form': form,
        'page_title': 'Add Subject'
    }
    if request.method == 'POST':
        if form.is_valid():
            name = form.cleaned_data.get('name')
            course = form.cleaned_data.get('course')
            staff = form.cleaned_data.get('staff')
            try:
                subject = Subject()
                subject.name = name
                subject.staff = staff
                subject.course = course
                subject.save()
                messages.success(request, "Successfully Added")
                return redirect(reverse('add_subject'))

            except Exception as e:
                messages.error(request, "Could Not Add " + str(e))
            else:
                messages.error(request, "Fill Form Properly")

    return render(request, 'hod_template/add_subject_template.html', context)

def manage_staff(request):
    allStaff = CustomUser.objects.filter(user_type=2)
    context = {
        'allStaff': allStaff,

```

```

        'page_title': 'Manage Staff'
    }
    return render(request, "hod_template/manage_staff.html", context)

def manage_student(request):
    students = CustomUser.objects.filter(user_type=3)
    context = {
        'students': students,
        'page_title': 'Manage Students'
    }
    return render(request, "hod_template/manage_student.html", context)

def manage_course(request):
    courses = Course.objects.all()
    context = {
        'courses': courses,
        'page_title': 'Manage Courses'
    }
    return render(request, "hod_template/manage_course.html", context)

def manage_subject(request):
    subjects = Subject.objects.all()
    context = {
        'subjects': subjects,
        'page_title': 'Manage Subjects'
    }
    return render(request, "hod_template/manage_subject.html", context)

def edit_staff(request, staff_id):
    staff = get_object_or_404(Staff, id=staff_id)
    form = StaffForm(request.POST or None, instance=staff)
    context = {
        'form': form,
        'staff_id': staff_id,
        'page_title': 'Edit Staff'
    }
    if request.method == 'POST':
        if form.is_valid():
            first_name = form.cleaned_data.get('first_name')
            last_name = form.cleaned_data.get('last_name')
            address = form.cleaned_data.get('address')
            username = form.cleaned_data.get('username')
            email = form.cleaned_data.get('email')
            gender = form.cleaned_data.get('gender')
            password = form.cleaned_data.get('password') or None

```

```

course = form.cleaned_data.get('course')
passport = request.FILES.get('profile_pic') or None
try:
    user = CustomUser.objects.get(id=staff.admin.id)
    user.username = username
    user.email = email
    if password != None:
        user.set_password(password)
    if passport != None:
        fs = FileSystemStorage()
        filename = fs.save(passport.name, passport)
        passport_url = fs.url(filename)
        user.profile_pic = passport_url
    user.first_name = first_name
    user.last_name = last_name
    user.gender = gender
    user.address = address
    staff.course = course
    user.save()
    staff.save()
    messages.success(request, "Successfully Updated")
    return redirect(reverse('edit_staff', args=[staff_id]))
except Exception as e:
    messages.error(request, "Could Not Update " + str(e))
else:
    messages.error(request, "Please fil form properly")
else:
    user = CustomUser.objects.get(id=staff_id)
    staff = Staff.objects.get(id=user.id)
    return render(request, "hod_template/edit_staff_template.html", context)

```

```

def edit_student(request, student_id):
    student = get_object_or_404(Student, id=student_id)
    form = StudentForm(request.POST or None, instance=student)
    context = {
        'form': form,
        'student_id': student_id,
        'page_title': 'Edit Student'
    }
    if request.method == 'POST':
        if form.is_valid():
            first_name = form.cleaned_data.get('first_name')
            last_name = form.cleaned_data.get('last_name')
            address = form.cleaned_data.get('address')
            username = form.cleaned_data.get('username')
            email = form.cleaned_data.get('email')
            gender = form.cleaned_data.get('gender')
            password = form.cleaned_data.get('password') or None

```

```

course = form.cleaned_data.get('course')
session = form.cleaned_data.get('session')
passport = request.FILES.get('profile_pic') or None
try:
    user = CustomUser.objects.get(id=student.admin.id)
    if passport != None:
        fs = FileSystemStorage()
        filename = fs.save(passport.name, passport)
        passport_url = fs.url(filename)
        user.profile_pic = passport_url
    user.username = username
    user.email = email
    if password != None:
        user.set_password(password)
    user.first_name = first_name
    user.last_name = last_name
    student.session = session
    user.gender = gender
    user.address = address
    student.course = course
    user.save()
    student.save()
    messages.success(request, "Successfully Updated")
    return redirect(reverse('edit_student', args=[student_id]))
except Exception as e:
    messages.error(request, "Could Not Update " + str(e))
else:
    messages.error(request, "Please Fill Form Properly!")
else:
    return render(request, "hod_template/edit_student_template.html", context)

```

```

def edit_course(request, course_id):
    instance = get_object_or_404(Course, id=course_id)
    form = CourseForm(request.POST or None, instance=instance)
    context = {
        'form': form,
        'course_id': course_id,
        'page_title': 'Edit Course'
    }
    if request.method == 'POST':
        if form.is_valid():
            name = form.cleaned_data.get('name')
            try:
                course = Course.objects.get(id=course_id)
                course.name = name
                course.save()
                messages.success(request, "Successfully Updated")
            except:

```

```

        messages.error(request, "Could Not Update")
    else:
        messages.error(request, "Could Not Update")

    return render(request, 'hod_template/edit_course_template.html', context)

def edit_subject(request, subject_id):
    instance = get_object_or_404(Subject, id=subject_id)
    form = SubjectForm(request.POST or None, instance=instance)
    context = {
        'form': form,
        'subject_id': subject_id,
        'page_title': 'Edit Subject'
    }
    if request.method == 'POST':
        if form.is_valid():
            name = form.cleaned_data.get('name')
            course = form.cleaned_data.get('course')
            staff = form.cleaned_data.get('staff')
            try:
                subject = Subject.objects.get(id=subject_id)
                subject.name = name
                subject.staff = staff
                subject.course = course
                subject.save()
                messages.success(request, "Successfully Updated")
                return redirect(reverse('edit_subject', args=[subject_id]))
            except Exception as e:
                messages.error(request, "Could Not Add " + str(e))
        else:
            messages.error(request, "Fill Form Properly")
    return render(request, 'hod_template/edit_subject_template.html', context)

def add_session(request):
    form = SessionForm(request.POST or None)
    context = {'form': form, 'page_title': 'Add Session'}
    if request.method == 'POST':
        if form.is_valid():
            try:
                form.save()
                messages.success(request, "Session Created")
                return redirect(reverse('add_session'))
            except Exception as e:
                messages.error(request, 'Could Not Add ' + str(e))
        else:
            messages.error(request, 'Fill Form Properly ')
    return render(request, "hod_template/add_session_template.html", context)

```

```

def manage_session(request):
    sessions = Session.objects.all()
    context = {'sessions': sessions, 'page_title': 'Manage Sessions'}
    return render(request, "hod_template/manage_session.html", context)

def edit_session(request, session_id):
    instance = get_object_or_404(Session, id=session_id)
    form = SessionForm(request.POST or None, instance=instance)
    context = {'form': form, 'session_id': session_id,
               'page_title': 'Edit Session'}
    if request.method == 'POST':
        if form.is_valid():
            try:
                form.save()
                messages.success(request, "Session Updated")
                return redirect(reverse('edit_session', args=[session_id]))
            except Exception as e:
                messages.error(
                    request, "Session Could Not Be Updated " + str(e))
                return render(request, "hod_template/edit_session_template.html", context)
        else:
            messages.error(request, "Invalid Form Submitted ")
            return render(request, "hod_template/edit_session_template.html", context)

    else:
        return render(request, "hod_template/edit_session_template.html", context)

@csrf_exempt
def check_email_availability(request):
    email = request.POST.get("email")
    try:
        user = CustomUser.objects.filter(email=email).exists()
        if user:
            return HttpResponse(True)
        return HttpResponse(False)
    except Exception as e:
        return HttpResponse(False)

@csrf_exempt
def student_feedback_message(request):
    if request.method != 'POST':
        feedbacks = FeedbackStudent.objects.all()
        context = {
            'feedbacks': feedbacks,

```

```

        'page_title': 'Student Feedback Messages'
    }
    return render(request, 'hod_template/student_feedback_template.html', context)
else:
    feedback_id = request.POST.get('id')
    try:
        feedback = get_object_or_404(FeedbackStudent, id=feedback_id)
        reply = request.POST.get('reply')
        feedback.reply = reply
        feedback.save()
        return HttpResponseRedirect(True)
    except Exception as e:
        return HttpResponseRedirect(False)

```

```

@csrf_exempt
def staff_feedback_message(request):
    if request.method != 'POST':
        feedbacks = FeedbackStaff.objects.all()
        context = {
            'feedbacks': feedbacks,
            'page_title': 'Staff Feedback Messages'
        }
        return render(request, 'hod_template/staff_feedback_template.html', context)
    else:
        feedback_id = request.POST.get('id')
        try:
            feedback = get_object_or_404(FeedbackStaff, id=feedback_id)
            reply = request.POST.get('reply')
            feedback.reply = reply
            feedback.save()
            return HttpResponseRedirect(True)
        except Exception as e:
            return HttpResponseRedirect(False)

```

```

@csrf_exempt
def view_staff_leave(request):
    if request.method != 'POST':
        allLeave = LeaveReportStaff.objects.all()
        context = {
            'allLeave': allLeave,
            'page_title': 'Leave Applications From Staff'
        }
        return render(request, "hod_template/staff_leave_view.html", context)
    else:
        id = request.POST.get('id')
        status = request.POST.get('status')
        if (status == '1'):

```

```

        status = 1
    else:
        status = -1
    try:
        leave = get_object_or_404(LeaveReportStaff, id=id)
        leave.status = status
        leave.save()
        return HttpResponseRedirect(True)
    except Exception as e:
        return False

```

```

@csrf_exempt
def view_student_leave(request):
    if request.method != 'POST':
        allLeave = LeaveReportStudent.objects.all()
        context = {
            'allLeave': allLeave,
            'page_title': 'Leave Applications From Students'
        }
        return render(request, "hod_template/student_leave_view.html", context)
    else:
        id = request.POST.get('id')
        status = request.POST.get('status')
        if (status == '1'):
            status = 1
        else:
            status = -1
        try:
            leave = get_object_or_404(LeaveReportStudent, id=id)
            leave.status = status
            leave.save()
            return HttpResponseRedirect(True)
        except Exception as e:
            return False

```

```

def admin_view_attendance(request):
    subjects = Subject.objects.all()
    sessions = Session.objects.all()
    context = {
        'subjects': subjects,
        'sessions': sessions,
        'page_title': 'View Attendance'
    }

    return render(request, "hod_template/admin_view_attendance.html", context)

```

```

@csrf_exempt
def get_admin_attendance(request):
    subject_id = request.POST.get('subject')
    session_id = request.POST.get('session')
    attendance_date_id = request.POST.get('attendance_date_id')
    try:
        subject = get_object_or_404(Subject, id=subject_id)
        session = get_object_or_404(Session, id=session_id)
        attendance = get_object_or_404(
            Attendance, id=attendance_date_id, session=session)
        attendance_reports = AttendanceReport.objects.filter(
            attendance=attendance)
        json_data = []
        for report in attendance_reports:
            data = {
                "status": str(report.status),
                "name": str(report.student)
            }
            json_data.append(data)
        return JsonResponse(json.dumps(json_data), safe=False)
    except Exception as e:
        return None

def admin_view_profile(request):
    admin = get_object_or_404(Admin, admin=request.user)
    form = AdminForm(request.POST or None, request.FILES or None,
        instance=admin)
    context = {'form': form,
        'page_title': 'View/Edit Profile'
    }
    if request.method == 'POST':
        try:
            if form.is_valid():
                first_name = form.cleaned_data.get('first_name')
                last_name = form.cleaned_data.get('last_name')
                password = form.cleaned_data.get('password') or None
                passport = request.FILES.get('profile_pic') or None
                custom_user = admin.admin
                if password != None:
                    custom_user.set_password(password)
                if passport != None:
                    fs = FileSystemStorage()
                    filename = fs.save(passport.name, passport)
                    passport_url = fs.url(filename)
                    custom_user.profile_pic = passport_url
                custom_user.first_name = first_name
                custom_user.last_name = last_name
                custom_user.save()

```

```

        messages.success(request, "Profile Updated!")
        return redirect(reverse('admin_view_profile'))
    else:
        messages.error(request, "Invalid Data Provided")
except Exception as e:
    messages.error(
        request, "Error Occured While Updating Profile " + str(e))
return render(request, "hod_template/admin_view_profile.html", context)

```

```

def admin_notify_staff(request):
    staff = CustomUser.objects.filter(user_type=2)
    context = {
        'page_title': "Send Notifications To Staff",
        'allStaff': staff
    }
    return render(request, "hod_template/staff_notification.html", context)

```

```

def admin_notify_student(request):
    student = CustomUser.objects.filter(user_type=3)
    context = {
        'page_title': "Send Notifications To Students",
        'students': student
    }
    return render(request, "hod_template/student_notification.html", context)

```

@csrf_exempt

```

def send_student_notification(request):
    id = request.POST.get('id')
    message = request.POST.get('message')
    student = get_object_or_404(Student, admin_id=id)
    try:
        url = "https://fcm.googleapis.com/fcm/send"
        body = {
            'notification': {
                'title': "Student Management System",
                'body': message,
                'click_action': reverse('student_view_notification'),
                'icon': static('dist/img/AdminLTELogo.png')
            },
            'to': student.admin.fcm_token
        }
        headers = {'Authorization':

```

```

'key=AAAA3Bm8j_M:APA91bElZIOLetwV696SoEtgzpjR2qbxBfxVBfDWFioPBWzfCfzQp2nRyC7_A2mluk
ZEHV4g1AmyC6P_HonvSkY2YylikT5tT3fe_1lrKod2Daigzhb2xnYQMxUWjCAIQcUexAMPZePB',
        'Content-Type': 'application/json'}

```

```

        data = requests.post(url, data=json.dumps(body), headers=headers)
        notification = NotificationStudent(student=student, message=message)
        notification.save()
        return HttpResponse("True")
    except Exception as e:
        return HttpResponse("False")

@csrf_exempt
def send_staff_notification(request):
    id = request.POST.get('id')
    message = request.POST.get('message')
    staff = get_object_or_404(Staff, admin_id=id)
    try:
        url = "https://fcm.googleapis.com/fcm/send"
        body = {
            'notification': {
                'title': "Student Management System",
                'body': message,
                'click_action': reverse('staff_view_notification'),
                'icon': static('dist/img/AdminLTELogo.png')
            },
            'to': staff.admin.fcm_token
        }
        headers = {'Authorization':
'key=AAAA3Bm8j_M:APA91bElZIOLetwV696SoEtgzpJr2qbxBfxVBfDWFiopBWzfCfzQp2nRyC7_A2mluk
ZEHV4g1AmyC6P_HonvSky2YyIKt5tT3fe_1lrKod2Daigzhb2xnYQMxUWjCAIQcUexAMPZePB',
'Content-Type': 'application/json'}
        data = requests.post(url, data=json.dumps(body), headers=headers)
        notification = NotificationStaff(staff=staff, message=message)
        notification.save()
        return HttpResponse("True")
    except Exception as e:
        return HttpResponse("False")

def delete_staff(request, staff_id):
    staff = get_object_or_404(CustomUser, staff__id=staff_id)
    staff.delete()
    messages.success(request, "Staff deleted successfully!")
    return redirect(reverse('manage_staff'))

def delete_student(request, student_id):
    student = get_object_or_404(CustomUser, student__id=student_id)
    student.delete()
    messages.success(request, "Student deleted successfully!")
    return redirect(reverse('manage_student'))

```

```

def delete_course(request, course_id):
    course = get_object_or_404(Course, id=course_id)
    try:
        course.delete()
        messages.success(request, "Course deleted successfully!")
    except Exception:
        messages.error(
            request, "Sorry, some students are assigned to this course already. Kindly change the affected
student course and try again")
    return redirect(reverse('manage_course'))

def delete_subject(request, subject_id):
    subject = get_object_or_404(Subject, id=subject_id)
    subject.delete()
    messages.success(request, "Subject deleted successfully!")
    return redirect(reverse('manage_subject'))

def delete_session(request, session_id):
    session = get_object_or_404(Session, id=session_id)
    try:
        session.delete()
        messages.success(request, "Session deleted successfully!")
    except Exception:
        messages.error(
            request, "There are students assigned to this session. Please move them to another session.")
    return redirect(reverse('manage_session'))

```

***** **staff-views.py** *****

```

import json

from django.contrib import messages
from django.core.files.storage import FileSystemStorage
from django.http import HttpResponseRedirect, JsonResponse
from django.shortcuts import (HttpResponseRedirect, get_object_or_404, redirect, render)
from django.urls import reverse
from django.views.decorators.csrf import csrf_exempt

from .forms import *
from .models import *
from . import forms, models
from datetime import date

def staff_home(request):

```

```

staff = get_object_or_404(Staff, admin=request.user)
total_students = Student.objects.filter(course=staff.course).count()
total_leave = LeaveReportStaff.objects.filter(staff=staff).count()
subjects = Subject.objects.filter(staff=staff)
total_subject = subjects.count()
attendance_list = Attendance.objects.filter(subject__in=subjects)
total_attendance = attendance_list.count()
attendance_list = []
subject_list = []
for subject in subjects:
    attendance_count = Attendance.objects.filter(subject=subject).count()
    subject_list.append(subject.name)
    attendance_list.append(attendance_count)
context = {
    'page_title': 'Staff Panel - ' + str(staff.admin.first_name) + ' ' + str(staff.admin.last_name[0]) + " + '
(' + str(staff.course) + ')",
    'total_students': total_students,
    'total_attendance': total_attendance,
    'total_leave': total_leave,
    'total_subject': total_subject,
    'subject_list': subject_list,
    'attendance_list': attendance_list
}
return render(request, 'staff_template/home_content.html', context)

def staff_take_attendance(request):
    staff = get_object_or_404(Staff, admin=request.user)
    subjects = Subject.objects.filter(staff_id=staff)
    sessions = Session.objects.all()
    context = {
        'subjects': subjects,
        'sessions': sessions,
        'page_title': 'Take Attendance'
    }

    return render(request, 'staff_template/staff_take_attendance.html', context)

@csrf_exempt
def get_students(request):
    subject_id = request.POST.get('subject')
    session_id = request.POST.get('session')
    try:
        subject = get_object_or_404(Subject, id=subject_id)
        session = get_object_or_404(Session, id=session_id)
        students = Student.objects.filter(
            course_id=subject.course.id, session=session)
        student_data = []
        for student in students:
            data = {

```

```

        "id": student.id,
        "name": student.admin.last_name + " " + student.admin.first_name
    }
    student_data.append(data)
    return JsonResponse(json.dumps(student_data), content_type='application/json', safe=False)
except Exception as e:
    return e

@csrf_exempt
def save_attendance(request):
    student_data = request.POST.get('student_ids')
    date = request.POST.get('date')
    subject_id = request.POST.get('subject')
    session_id = request.POST.get('session')
    students = json.loads(student_data)
    try:
        session = get_object_or_404(Session, id=session_id)
        subject = get_object_or_404(Subject, id=subject_id)
        attendance = Attendance(session=session, subject=subject, date=date)
        attendance.save()

        for student_dict in students:
            student = get_object_or_404(Student, id=student_dict.get('id'))
            attendance_report = AttendanceReport(student=student, attendance=attendance,
status=student_dict.get('status'))
            attendance_report.save()
    except Exception as e:
        return None

    return HttpResponse("OK")

def staff_update_attendance(request):
    staff = get_object_or_404(Staff, admin=request.user)
    subjects = Subject.objects.filter(staff_id=staff)
    sessions = Session.objects.all()
    context = {
        'subjects': subjects,
        'sessions': sessions,
        'page_title': 'Update Attendance'
    }

    return render(request, 'staff_template/staff_update_attendance.html', context)

@csrf_exempt
def get_student_attendance(request):
    attendance_date_id = request.POST.get('attendance_date_id')
    try:
        date = get_object_or_404(Attendance, id=attendance_date_id)
        attendance_data = AttendanceReport.objects.filter(attendance=date)

```

```

        student_data = []
        for attendance in attendance_data:
            data = {"id": attendance.student.admin.id,
                    "name": attendance.student.admin.last_name + " " +
attendance.student.admin.first_name,
                    "status": attendance.status}
            student_data.append(data)
        return JsonResponse(json.dumps(student_data), content_type='application/json', safe=False)
    except Exception as e:
        return e

```

@csrf_exempt

```

def update_attendance(request):
    student_data = request.POST.get('student_ids')
    date = request.POST.get('date')
    students = json.loads(student_data)
    try:
        attendance = get_object_or_404(Attendance, id=date)

        for student_dict in students:
            student = get_object_or_404(
                Student, admin_id=student_dict.get('id'))
            attendance_report = get_object_or_404(AttendanceReport, student=student,
attendance=attendance)
            attendance_report.status = student_dict.get('status')
            attendance_report.save()
    except Exception as e:
        return None

```

```

    return HttpResponse("OK")

```

```

def staff_apply_leave(request):
    form = LeaveReportStaffForm(request.POST or None)
    staff = get_object_or_404(Staff, admin_id=request.user.id)
    context = {
        'form': form,
        'leave_history': LeaveReportStaff.objects.filter(staff=staff),
        'page_title': 'Apply for Leave'
    }
    if request.method == 'POST':
        if form.is_valid():
            try:
                obj = form.save(commit=False)
                obj.staff = staff
                obj.save()
                messages.success(
                    request, "Application for leave has been submitted for review")
                return redirect(reverse('staff_apply_leave'))
            except Exception:

```

```

        messages.error(request, "Could not apply!")
    else:
        messages.error(request, "Form has errors!")
    return render(request, "staff_template/staff_apply_leave.html", context)

def staff_feedback(request):
    form = FeedbackStaffForm(request.POST or None)
    staff = get_object_or_404(Staff, admin_id=request.user.id)
    context = {
        'form': form,
        'feedbacks': FeedbackStaff.objects.filter(staff=staff),
        'page_title': 'Add Feedback'
    }
    if request.method == 'POST':
        if form.is_valid():
            try:
                obj = form.save(commit=False)
                obj.staff = staff
                obj.save()
                messages.success(request, "Feedback submitted for review")
                return redirect(reverse('staff_feedback'))
            except Exception:
                messages.error(request, "Could not Submit!")
        else:
            messages.error(request, "Form has errors!")
    return render(request, "staff_template/staff_feedback.html", context)

def staff_view_profile(request):
    staff = get_object_or_404(Staff, admin=request.user)
    form = StaffEditForm(request.POST or None, request.FILES or None, instance=staff)
    context = {'form': form, 'page_title': 'View/Update Profile'}
    if request.method == 'POST':
        try:
            if form.is_valid():
                first_name = form.cleaned_data.get('first_name')
                last_name = form.cleaned_data.get('last_name')
                password = form.cleaned_data.get('password') or None
                address = form.cleaned_data.get('address')
                gender = form.cleaned_data.get('gender')
                passport = request.FILES.get('profile_pic') or None
                admin = staff.admin
                if password != None:
                    admin.set_password(password)
                if passport != None:
                    fs = FileSystemStorage()
                    filename = fs.save(passport.name, passport)
                    passport_url = fs.url(filename)
                    admin.profile_pic = passport_url
                admin.first_name = first_name

```

```

        admin.last_name = last_name
        admin.address = address
        admin.gender = gender
        admin.save()
        staff.save()
        messages.success(request, "Profile Updated!")
        return redirect(reverse('staff_view_profile'))
    else:
        messages.error(request, "Invalid Data Provided")
        return render(request, "staff_template/staff_view_profile.html", context)
except Exception as e:
    messages.error(
        request, "Error Occured While Updating Profile " + str(e))
    return render(request, "staff_template/staff_view_profile.html", context)

return render(request, "staff_template/staff_view_profile.html", context)

@csrf_exempt
def staff_fcmtoken(request):
    token = request.POST.get('token')
    try:
        staff_user = get_object_or_404(CustomUser, id=request.user.id)
        staff_user.fcm_token = token
        staff_user.save()
        return HttpResponse("True")
    except Exception as e:
        return HttpResponse("False")

def staff_view_notification(request):
    staff = get_object_or_404(Staff, admin=request.user)
    notifications = NotificationStaff.objects.filter(staff=staff)
    context = {
        'notifications': notifications,
        'page_title': "View Notifications"
    }
    return render(request, "staff_template/staff_view_notification.html", context)

def staff_add_result(request):
    staff = get_object_or_404(Staff, admin=request.user)
    subjects = Subject.objects.filter(staff=staff)
    sessions = Session.objects.all()
    context = {
        'page_title': 'Result Upload',
        'subjects': subjects,
        'sessions': sessions
    }
    if request.method == 'POST':
        try:
            student_id = request.POST.get('student_list')

```

```

subject_id = request.POST.get('subject')
test = request.POST.get('test')
exam = request.POST.get('exam')
student = get_object_or_404(Student, id=student_id)
subject = get_object_or_404(Subject, id=subject_id)
try:
    data = StudentResult.objects.get(
        student=student, subject=subject)
    data.exam = exam
    data.test = test
    data.save()
    messages.success(request, "Scores Updated")
except:
    result = StudentResult(student=student, subject=subject, test=test, exam=exam)
    result.save()
    messages.success(request, "Scores Saved")
except Exception as e:
    messages.warning(request, "Error Occured While Processing Form")
return render(request, "staff_template/staff_add_result.html", context)

```

@csrf_exempt

```

def fetch_student_result(request):
    try:
        subject_id = request.POST.get('subject')
        student_id = request.POST.get('student')
        student = get_object_or_404(Student, id=student_id)
        subject = get_object_or_404(Subject, id=subject_id)
        result = StudentResult.objects.get(student=student, subject=subject)
        result_data = {
            'exam': result.exam,
            'test': result.test
        }
        return JsonResponse(json.dumps(result_data))
    except Exception as e:
        return JsonResponse('False')

```

#library

```

def add_book(request):
    if request.method == "POST":
        name = request.POST['name']
        author = request.POST['author']
        isbn = request.POST['isbn']
        category = request.POST['category']

        books = Book.objects.create(name=name, author=author, isbn=isbn, category=category )
        books.save()
        alert = True
        return render(request, "staff_template/add_book.html", {'alert':alert})
    context = {

```

```

        'page_title': "Add Book"
    }
    return render(request, "staff_template/add_book.html",context)

#issue book

def issue_book(request):
    form = forms.IssueBookForm()
    if request.method == "POST":
        form = forms.IssueBookForm(request.POST)
        if form.is_valid():
            obj = models.IssuedBook()
            obj.student_id = request.POST['name2']
            obj.isbn = request.POST['isbn2']
            obj.save()
            alert = True
            return render(request, "staff_template/issue_book.html", {'obj':obj, 'alert':alert})
    return render(request, "staff_template/issue_book.html", {'form':form})

def view_issued_book(request):
    issuedBooks = IssuedBook.objects.all()
    details = []
    for i in issuedBooks:
        days = (date.today()-i.issued_date)
        d=days.days
        fine=0
        if d>14:
            day=d-14
            fine=day*5
        books = list(models.Book.objects.filter(isbn=i.isbn))
        # students = list(models.Student.objects.filter(admin=i.admin))
        i=0
        for l in books:
            t=(books[i].name,books[i].isbn,issuedBooks[0].issued_date,issuedBooks[0].expiry_date,fine)
            i=i+1
            details.append(t)
    return render(request, "staff_template/view_issued_book.html", {'issuedBooks':issuedBooks,
'details':details})

```

***** Student-views.py *****

```

import json
import math
from datetime import datetime

from django.contrib import messages
from django.core.files.storage import FileSystemStorage
from django.http import HttpResponse, JsonResponse

```

```

from django.shortcuts import (HttpResponseRedirect, get_object_or_404,
                             redirect, render)
from django.urls import reverse
from django.views.decorators.csrf import csrf_exempt

from .forms import *
from .models import *

def student_home(request):
    student = get_object_or_404(Student, admin=request.user)
    total_subject = Subject.objects.filter(course=student.course).count()
    total_attendance = AttendanceReport.objects.filter(student=student).count()
    total_present = AttendanceReport.objects.filter(student=student, status=True).count()
    if total_attendance == 0: # Don't divide. DivisionByZero
        percent_absent = percent_present = 0
    else:
        percent_present = math.floor((total_present/total_attendance) * 100)
        percent_absent = math.ceil(100 - percent_present)
    subject_name = []
    data_present = []
    data_absent = []
    subjects = Subject.objects.filter(course=student.course)
    for subject in subjects:
        attendance = Attendance.objects.filter(subject=subject)
        present_count = AttendanceReport.objects.filter(
            attendance__in=attendance, status=True, student=student).count()
        absent_count = AttendanceReport.objects.filter(
            attendance__in=attendance, status=False, student=student).count()
        subject_name.append(subject.name)
        data_present.append(present_count)
        data_absent.append(absent_count)
    context = {
        'total_attendance': total_attendance,
        'percent_present': percent_present,
        'percent_absent': percent_absent,
        'total_subject': total_subject,
        'subjects': subjects,
        'data_present': data_present,
        'data_absent': data_absent,
        'data_name': subject_name,
        'page_title': 'Student Homepage'
    }
    return render(request, 'student_template/home_content.html', context)

@csrf_exempt
def student_view_attendance(request):
    student = get_object_or_404(Student, admin=request.user)
    if request.method != 'POST':

```

```

course = get_object_or_404(Course, id=student.course.id)
context = {
    'subjects': Subject.objects.filter(course=course),
    'page_title': 'View Attendance'
}
return render(request, 'student_template/student_view_attendance.html', context)
else:
    subject_id = request.POST.get('subject')
    start = request.POST.get('start_date')
    end = request.POST.get('end_date')
    try:
        subject = get_object_or_404(Subject, id=subject_id)
        start_date = datetime.strptime(start, "%Y-%m-%d")
        end_date = datetime.strptime(end, "%Y-%m-%d")
        attendance = Attendance.objects.filter(
            date__range=(start_date, end_date), subject=subject)
        attendance_reports = AttendanceReport.objects.filter(
            attendance__in=attendance, student=student)
        json_data = []
        for report in attendance_reports:
            data = {
                "date": str(report.attendance.date),
                "status": report.status
            }
            json_data.append(data)
        return JsonResponse(json.dumps(json_data), safe=False)
    except Exception as e:
        return None

```

```

def student_apply_leave(request):
    form = LeaveReportStudentForm(request.POST or None)
    student = get_object_or_404(Student, admin_id=request.user.id)
    context = {
        'form': form,
        'leave_history': LeaveReportStudent.objects.filter(student=student),
        'page_title': 'Apply for leave'
    }
    if request.method == 'POST':
        if form.is_valid():
            try:
                obj = form.save(commit=False)
                obj.student = student
                obj.save()
                messages.success(
                    request, "Application for leave has been submitted for review")
                return redirect(reverse('student_apply_leave'))
            except Exception:
                messages.error(request, "Could not submit")
        else:

```

```

        messages.error(request, "Form has errors!")
    return render(request, "student_template/student_apply_leave.html", context)

def student_feedback(request):
    form = FeedbackStudentForm(request.POST or None)
    student = get_object_or_404(Student, admin_id=request.user.id)
    context = {
        'form': form,
        'feedbacks': FeedbackStudent.objects.filter(student=student),
        'page_title': 'Student Feedback'
    }
    if request.method == 'POST':
        if form.is_valid():
            try:
                obj = form.save(commit=False)
                obj.student = student
                obj.save()
                messages.success(
                    request, "Feedback submitted for review")
                return redirect(reverse('student_feedback'))
            except Exception:
                messages.error(request, "Could not Submit!")
        else:
            messages.error(request, "Form has errors!")
    return render(request, "student_template/student_feedback.html", context)

def student_view_profile(request):
    student = get_object_or_404(Student, admin=request.user)
    form = StudentEditForm(request.POST or None, request.FILES or None,
                           instance=student)
    context = {'form': form,
               'page_title': 'View/Edit Profile'
               }
    if request.method == 'POST':
        try:
            if form.is_valid():
                first_name = form.cleaned_data.get('first_name')
                last_name = form.cleaned_data.get('last_name')
                password = form.cleaned_data.get('password') or None
                address = form.cleaned_data.get('address')
                gender = form.cleaned_data.get('gender')
                passport = request.FILES.get('profile_pic') or None
                admin = student.admin
                if password != None:
                    admin.set_password(password)
                if passport != None:
                    fs = FileSystemStorage()
                    filename = fs.save(passport.name, passport)

```

```

        passport_url = fs.url(filename)
        admin.profile_pic = passport_url
        admin.first_name = first_name
        admin.last_name = last_name
        admin.address = address
        admin.gender = gender
        admin.save()
        student.save()
        messages.success(request, "Profile Updated!")
        return redirect(reverse('student_view_profile'))
    else:
        messages.error(request, "Invalid Data Provided")
except Exception as e:
    messages.error(request, "Error Occured While Updating Profile " + str(e))

return render(request, "student_template/student_view_profile.html", context)

@csrf_exempt
def student_fcmtoken(request):
    token = request.POST.get('token')
    student_user = get_object_or_404(CustomUser, id=request.user.id)
    try:
        student_user.fcm_token = token
        student_user.save()
        return HttpResponse("True")
    except Exception as e:
        return HttpResponse("False")

def student_view_notification(request):
    student = get_object_or_404(Student, admin=request.user)
    notifications = NotificationStudent.objects.filter(student=student)
    context = {
        'notifications': notifications,
        'page_title': "View Notifications"
    }
    return render(request, "student_template/student_view_notification.html", context)

def student_view_result(request):
    student = get_object_or_404(Student, admin=request.user)
    results = StudentResult.objects.filter(student=student)
    context = {
        'results': results,
        'page_title': "View Results"
    }
    return render(request, "student_template/student_view_result.html", context)

#library

def view_books(request):

```

```
books = Book.objects.all()
context = {
    'books': books,
    'page_title': "Library"
}
return render(request, "student_template/view_books.html", context)
```

8. Limitations And Future Enhancements

Limitations

While *CampusConnect: College ERP Portal* is a robust solution for educational institutions, certain limitations exist in its current iteration:

1. Limited Scalability:
 - The system is optimized for medium-sized institutions. However, larger institutions with significantly more data and concurrent users may require additional infrastructure upgrades.
2. Internet Dependency:
 - The portal requires a stable internet connection for all functionalities. Offline access to critical data is not currently supported.
3. Customization Challenges:
 - Although the portal offers many features, it lacks the flexibility for institutions to add custom workflows or unique modules directly without developer intervention.
4. User Adoption:
 - For staff or students unfamiliar with technology, adapting to the system may require training and support.
5. Third-Party Integration Limitations:
 - Integration with external services such as Learning Management Systems (LMS), payment gateways, and cloud-based storage systems is not comprehensive.
6. Mobile Experience:
 - While the portal is responsive and accessible via web browsers, a dedicated mobile application for Android and iOS is not yet available.
7. Report Customization:
 - Currently, the report generation feature is predefined and lacks extensive customization options for creating specific or unique reports.
8. Data Backup and Recovery:
 - While the portal supports manual backups, automated and incremental

backup options are not implemented in this version.

Future Enhancements

To address the current limitations and stay competitive, *CampusConnect* plans the following enhancements in future iterations:

1. Scalability Enhancements:

- Implement microservices architecture and cloud-based scaling solutions to support larger institutions seamlessly.

2. Offline Mode:

- Enable offline access for students and staff to view key data, such as attendance records, schedules, and notifications, with automatic synchronization when back online.

3. Dedicated Mobile Application:

- Develop standalone mobile apps for Android and iOS with features like push notifications, offline access, and real-time updates.

4. Advanced Report Generation:

- Introduce customizable reporting tools that allow users to define specific parameters and generate detailed, dynamic reports.

5. Third-Party Integrations:

- Provide seamless integration with popular LMS platforms (like Moodle), payment gateways, and Google Drive for extended functionality.

6. AI-Driven Insights:

- Introduce AI-powered analytics to offer actionable insights on student performance, resource utilization, and predictive enrollment trends.

7. Enhanced Security Measures:

- Integrate advanced security features like multi-factor authentication, biometric login, and real-time monitoring for data breaches.

8. Automated Data Backups:

- Develop a secure, automated backup system with version control for improved data recovery and reliability.

9. User-Friendly Onboarding:

- Provide an interactive onboarding system with tutorials and step-by-step guides to help new users understand the platform efficiently.

10. Gamification of Student Activities:

- Add gamified elements like leaderboards, badges, and achievement tracking to increase student engagement.

9. conclusion

The development of *CampusConnect: College ERP Portal* highlights the potential of technology to revolutionize educational management. This portal serves as a centralized system for handling key institutional processes, including student and staff data management, attendance tracking, course scheduling, and notifications. By leveraging the Django framework for backend development, PostgreSQL for database management, and Bootstrap for responsive design, *CampusConnect* ensures scalability, security, and ease of use. The incorporation of REST APIs for seamless communication further strengthens its functionality.

The project successfully addresses the core requirements of educational institutions, such as data centralization, secure access, and streamlined operations. However, limitations like scalability constraints, lack of offline access, and minimal third-party integration suggest areas for improvement.

Future enhancements, such as AI-powered analytics, dedicated mobile applications, and offline access, will solidify *CampusConnect* as an essential tool for modern educational institutions. These upgrades will make the system more adaptive, scalable, and user-friendly.

In conclusion, *CampusConnect* is a forward-thinking solution that sets the foundation for digitized education management. It is poised to evolve into a comprehensive ERP platform that meets the dynamic needs of institutions while maintaining a focus on security, reliability, and innovation.

10. Bibliography

- The following references were utilized during the design, development, and documentation of *CampusConnect: College ERP Portal*:

Books

- Silberschatz, A., Korth, H., & Sudarshan, S. (2019). *Database System Concepts*. McGraw-Hill Education.
- Sommerville, I. (2016). *Software Engineering*. Pearson Education.

Web Resources

- Django Official Documentation: <https://docs.djangoproject.com/>
- PostgreSQL Official Documentation: <https://www.postgresql.org/docs/>
- Bootstrap Framework: <https://getbootstrap.com/>
- RESTful API Guidelines: <https://restfulapi.net/>

Research Papers

- IEEE Papers on ERP System Design and Implementation.
- Case Studies on Education ERP Systems from ResearchGate.

Online Tutorials and Forums

- Stack Overflow: <https://stackoverflow.com/>
- W3Schools Tutorials: <https://www.w3schools.com/>
- TutorialsPoint: <https://www.tutorialspoint.com/>